



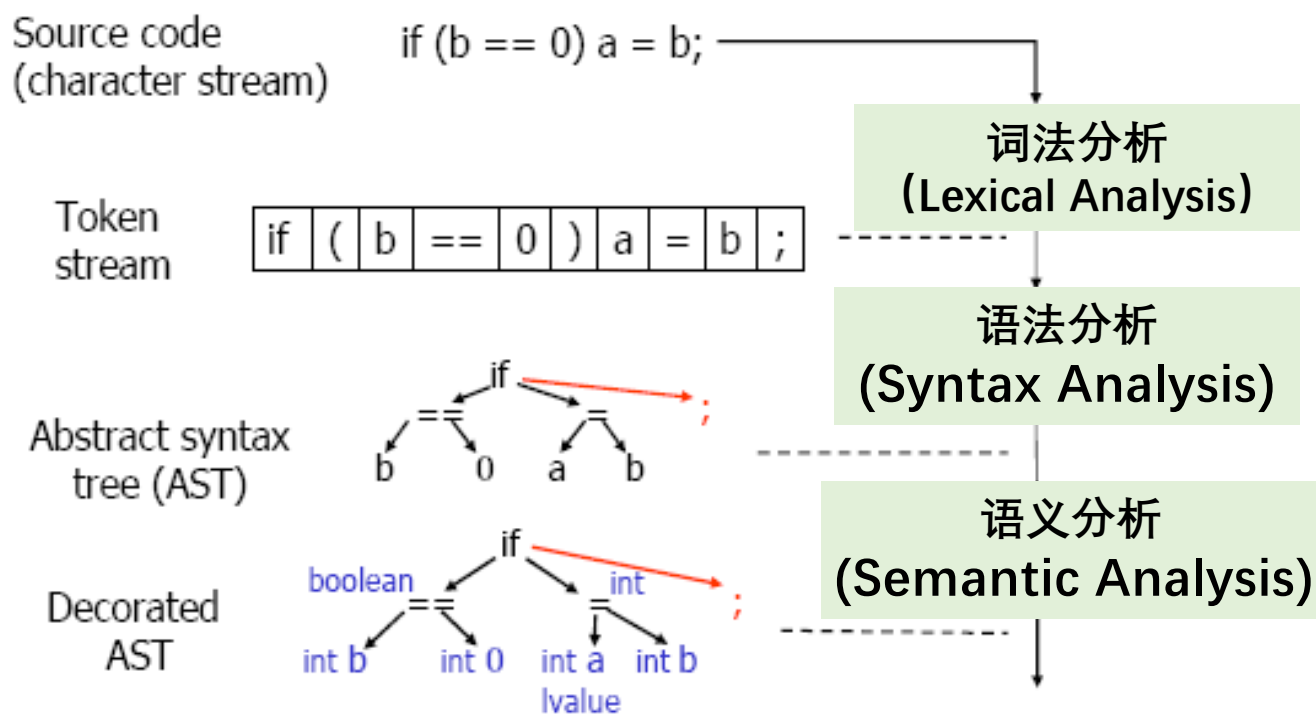
编译原理

--词法分析自动生成

刘爽

中国人民大学信息学院

■ 编译器结构

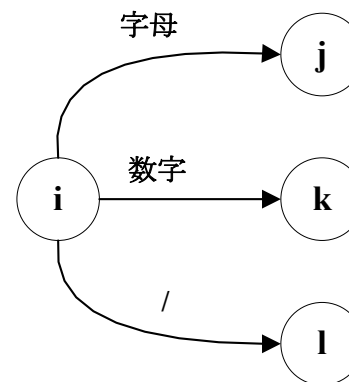


■ 状态转换图的实现

- 程序实现:每个状态结点对应一段程序.

1) 不含回路的分叉结点:

- CASE 或者 IF--THEN—ELSE



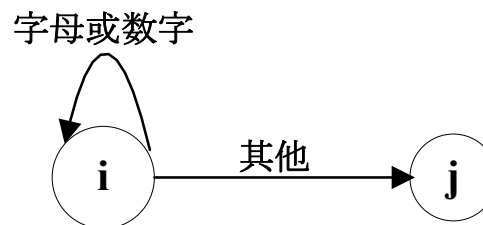
```
GetChar()
if (IsLetter())
    {状态j的对应程序段}
else if (IsDigit())
    {状态k的对应程序段}
else if (ch == '/')
    {状态l的对应程序段}
else
    {错误处理}
```

2) 含回路的分叉结:

- WHILE

3) 终点结:

- RETURN(Code, Value): 返回调用者



```
GetChar();
while (IsLetter() or IsDigit())
    GetChar();
    状态j的对应程序段
```

Outline

- 词法分析器的作用
- 词法分析程序的设计与实现
 - 状态转换图
- 正则表达式和有限自动机
 - 正规/正则表达式 (Regular Expression)
 - 有限自动机 (Finite Automata)
 - 正则文法 (Regular Grammar)
- 词法分析程序的自动生成

问题的提出

- 将状态转换图的概念稍加形式化，更好的构造词法分析程序
- 通过正则表达式与有限自动机的相互转换，实现词法分析程序的自动生成器。
- 有助于理解词法分析器的生成器，可以自动产生有效的词法分析器。（例如 lex, flex, Jlex）

■ 词法分析器的实现步骤

- 用正则表达式描述词法规则，设置单词种别和属性；
- 按照右图所示步骤，逐步实现词法分析器；



正则表达式与正则集 (Regular Set)

- **正则表达式**可以详细说明单词符号的结构，可以精确地定义集合（**正则集合**）。**正则表达式**和**正则集合**的递归定义如下：

1. ε 和 Φ 都是 Σ 上的**正则式**，其**正则集**为 $\{\varepsilon\}$ 和 Φ ；
2. 任何 $a \in \Sigma$ ， a 是 Σ 的一个**正则式**，**正则集**为 $\{a\}$ ；
3. 假定 U 和 V 都是 Σ 上的**正则式**，**正则集**为 $L(U)$ 和 $L(V)$ ，那么，

$(U|V)$, (UV) , $(U)^*$ 也都是 Σ 上的**正则式**，**正则集**为 $L(U) \cup L(V)$ ， $L(U)L(V)$ 和 $L(U)^*$ ；

仅由有限次使用上述三步骤而定义的表达式才是 Σ 上的**正则表达式**，仅

由这些**正则表达式**所表示的字集才是 Σ 上的**正则集**。

正则集合（正规集）：可以用一个正则表达式定义的语言。

正则表达式和正则集的例子

令 $\Sigma = \{a, b\}$,

正则表达式

a

$a \mid b$

ab

$(a \mid b)(a \mid b)$

a^*

$(a \mid b)^*$

$(a \mid b)^*(aa \mid bb)(a \mid b)^*$

正则集合集

$\{a\}$

$\{a, b\}$

$\{ab\}$

$\{aa, ab, ba, bb\}$

$\{\varepsilon, a, aa, \dots \text{任意个 } a \text{ 的串}\}$

$\{\varepsilon, a, b, aa, ab, \dots \text{所有由 } a \text{ 和 } b \text{ 组成的串}\}$

$\{\Sigma^* \text{ 上所有含有两个相继的 } a \text{ 或两个相继的 } b \text{ 组成的串}\}$

正则表达式和正则集—练习

- $\Sigma = \{a, b\}$ 则 Σ 上的正则表达式
 - $a^*ba^*ba^*ba^*$ 表示的正则集是?
 - $((\varepsilon|a)b^*)^*$ 表示的正则集是?

正则表达式的等价性

- 若两个正则表达式所表示的正则集合相同，则认为二者等价。两个等价的正则表达式U和V记为 $U=V$ 。
 - 例如： $U = (a \mid b)$, $V = (b \mid a)$
 - 又如： $U = b(ab)^*$, $V = (ba)^*b$
 - 再如： $U = (a \mid b)^*$, $V = (a^* \mid b^*)^*$

正则表达式的性质

- 设U、V和W都是正则表达式，则如下关系普遍成立：
 - $U|V = V|U$ (交换律)
 - $U|(V|W) = (U|V)|W$ (结合律)
 - $U(VW) = (UV)W$ (结合律)
 - $U(V|W) = UV | UW$ (分配律)
 - $(V|W)U = VU | WU$;
 - $\varepsilon U = U\varepsilon = U$
 - $r|r = r \quad r^* = \varepsilon | r | rr | \dots$ “或”的抽取律
- 运算优先级和结合性:
 - * 高于 "连接" 高于 |
 - | 具有交换律、结合律
 - “连接” 具有结合律、分配律
 - () 指定优先关系

■ Outline

- 词法分析器的作用
- 词法分析程序的设计与实现
 - 状态转换图
- 正则表达式和有限自动机
 - 正规/正则表达式 (Regular Expression)
 - 有限自动机 (Finite Automata)
 - 正则文法 (Regular Grammar)
- 词法分析程序的自动生成

有限自动机

有限自动机

- **有限自动机**（也称有穷自动机）作为一种**识别**装置，它能准确地识别**正则集合**，即识别**正则文法所定义的语言与正则表达式所表示的集合**，引入有穷自动机这个理论，正是为词法分析程序的自动构造寻找特殊的方法和工具。
- **有限自动机**分为两类：
 - **确定有限自动机** Deterministic Finite Automata (DFA)
 - **不确定有限自动机** Nondeterministic Finite Automata (NFA)
- 其中“不确定”的含义是：对于**某个输入符号**，在**同一状态**上存在**不止一种转换**。
- 确定和不确定有限自动机都能而且仅能识别**正则集合**，即它们能够识别正则表达式所表示的语言。

确定有限自动机

确定有限自动机 (DFA)

- 一个确定的有限自动机(DFA)是一个五元式: $M=(S, \Sigma, \delta, s_0, F)$ 其中
 - S 是一个有限集, 它的每个元素称为一个状态。
 - Σ 是一个有穷字母表, 它的每个元素称为一个输入字符
 - $\delta: S \times \Sigma \rightarrow S$ 是一个单值映射 (确定性)。
 - $\delta(s, a)=s'$ 意味着: 当现行状态为 s 、输入字符为 a 时, 将转换到下一个状态 s' 。
我们称 s' 为 s 的一个后继状态。
 - $s_0 \in S$, 是唯一的初态。
 - $F \subseteq S$, 是一个终态集 (可空)

确定的有限自动机—表示方式

DFA $M = (\{S, U, V, Q\}, \{a, b\}, \delta, S, \{Q\})$ 其中 δ 定义为:

$\delta(S, a) = U$ $\delta(V, a) = U$
 $\delta(S, b) = V$ $\delta(V, b) = Q$
 $\delta(U, a) = Q$ $\delta(Q, a) = Q$
 $\delta(U, b) = V$ $\delta(Q, b) = Q$

状态 \ 字符	a	b
S	U	V
U	Q	V
V	U	Q
Q	Q	Q

图 (1) 状态转换矩阵

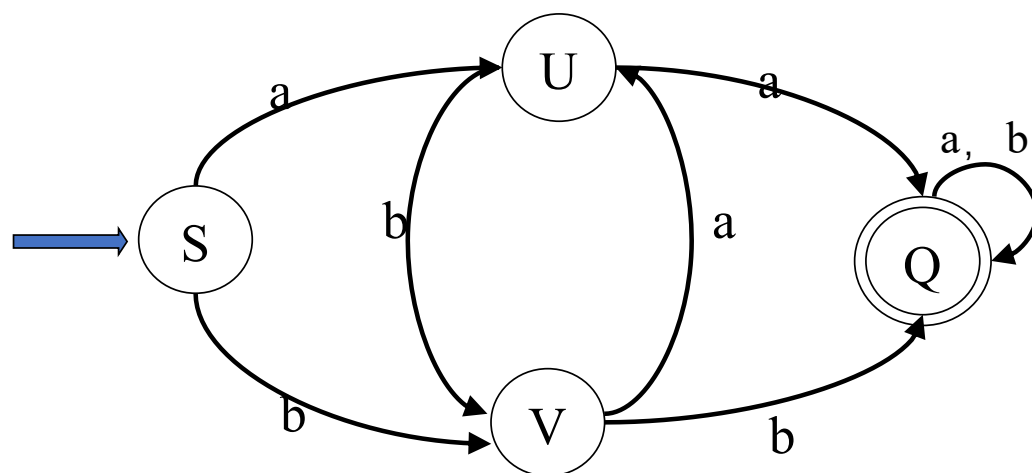


图 (2) 状态转换图

$\delta(s, a)$ 值-----状态转换矩阵值。如图 (1) 所示: 行--状态, 列--输入字符

一个DFA也可以表示为一张 (确定的) 状态转换图, 如图 (2) 所示

练习

DFA M 可用一张（确定的）状态转换图表示。

• DFA $M = (\{A, B, C, D\}, \{a, b\}, \delta, A, \{D\})$

$$\delta(A, a) = B \quad \delta(A, b) = A$$

$$\delta(B, a) = B \quad \delta(B, b) = C$$

$$\delta(C, a) = B \quad \delta(C, b) = D$$

$$\delta(D, a) = B \quad \delta(D, b) = A$$

画出其对应的状态转换图和状态转换矩阵

答案

$M(A,a)=B$ $M(A,b)=A$
 $M(B,a)=B$ $M(B,b)=C$
 $M(C,a)=B$ $M(C,b)=D$
 $M(D,a)=B$ $M(D,b)=A$

状态 \ 字符	a	b
A	B	A
B	B	C
C	B	D
D	B	A

图 (1) 状态转换矩阵

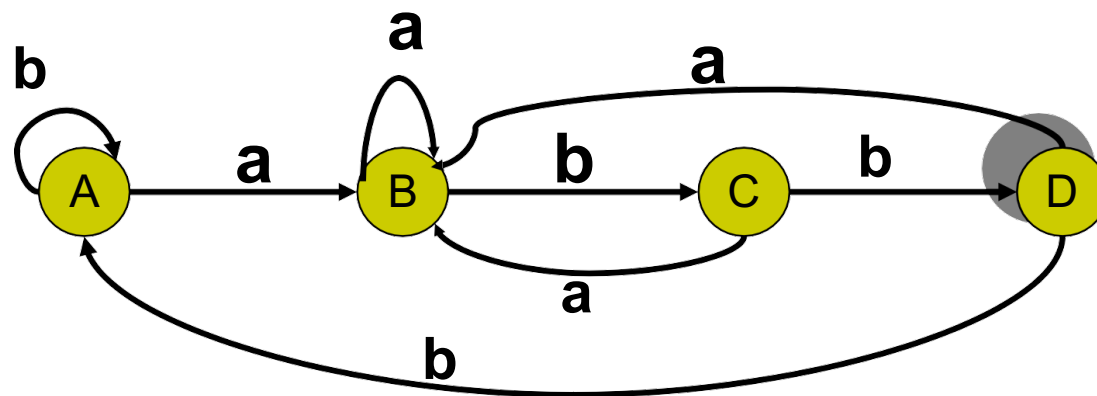


图 (2) 状态转图

确定有限自动机 (DFA) 接受的字符串

- DFA接受的字符串
 - 对于 Σ^* 中的任何字符串 t , 若存在一条从初态到某一终态的路径, 且这条路径上所有弧的标记字符连接成的字符串等于 t , (即若 $t = t_1 t_2 \dots t_n \in \Sigma^*$, $\delta(S, t_1) = P_1$, $\delta(P_1, t_2) = P_2$, ..., $\delta(P_{n-1}, t_n) = P_n$, 其中 S 为 M 的开始状态, $P_n \in Z$, Z 为终态集) 则称 t 可被 **DFA M** 接受(识别)
 - 若 M 的初态同时又是终态, 则空字符串可为 M 所识别。
 - 一个 **DFA M** 所能识别的所有的字符串的集合记为: **$L(M)$**
 - Σ 上的一个字集 **$V \subseteq \Sigma^*$** 是正则的, 当且仅当存在 Σ 上的 **DFA M** , 使得 **$V = L(M)$**
- Σ^* 上的符号串 t , (我们将它表示成 $t_1 t_x$ 的形式, 其中 $t_1 \in \Sigma$, $t_x \in \Sigma^*$) 在 **DFA M** 上运行的定义为:
 - $\delta(Q, t_1 t_x) = \delta(\delta(Q, t_1), t_x)$ 其中 $Q \in S$ (是 M 的开始状态)

确定有限自动机—例子

- 例：证明 $t=baab$ 被例中的DFA所接受。

$$\begin{aligned}\delta(S, baab) \\&= \delta(\delta(S, b), aab) \\&= \delta(V, aab) \\&= \delta(\delta(V, a), ab) \\&= \delta(U, ab) \\&= \delta(f(U, a), b) \\&= \delta(Q, b) \\&= Q\end{aligned}$$

DFA $M = (\{S, U, V, Q\}, \{a, b\}, \delta, S, \{Q\})$ 其中 δ 定义为：

$$\begin{array}{ll}\delta(S, a) = U & \delta(V, a) = U \\ \delta(S, b) = V & \delta(V, b) = Q \\ \delta(U, a) = Q & \delta(Q, a) = Q \\ \delta(U, b) = V & \delta(Q, b) = Q\end{array}$$

Q属于终态, 找到了从M的初态到终态的一条路径, 该路径上的字符连接成的字符串为 t 。

非确定有限自动机

非确定有限自动机 (NFA)

- 一个非确定有限自动机(NFA)是一个五元式: $M=(S, \Sigma, \delta, S_0, F)$
其中
 - S 是一个有限集, 它的每个元素称为一个状态。
 - Σ 是一个有穷字母表, 它的每个元素称为一个输入字符
 - $\delta: S \times \Sigma^* \rightarrow 2^S$ 是一个从 $S \times \Sigma^*$ 至 S 子集的单值映射。
 - $S_0 \subseteq S$, 是一个非空的初态集
 - $F \subseteq S$, 是一个终态集 (可空)

若 δ 是一个多值函数, 且输入可允许为 ε , 则有穷自动机是不确定的, 即在某个状态下, 对于某个输入字符存在多个后继状态。

非确定的有限自动机—表示方式

NFA $N = (\{S, P, Z\}, \{0, 1\}, \delta, \{S, P\}, \{Z\})$

- $\delta(S, 0) = \{P\}$
- $\delta(S, 1) = \{S, Z\}$
- $\delta(P, 1) = \{Z\}$
- $\delta(Z, 0) = \{P\}$
- $\delta(Z, 1) = \{P\}$

	0	1
S	{P}	{S, Z}
P	{}	{Z}
Z	{P}	{P}

图 (1) 状态转换矩阵

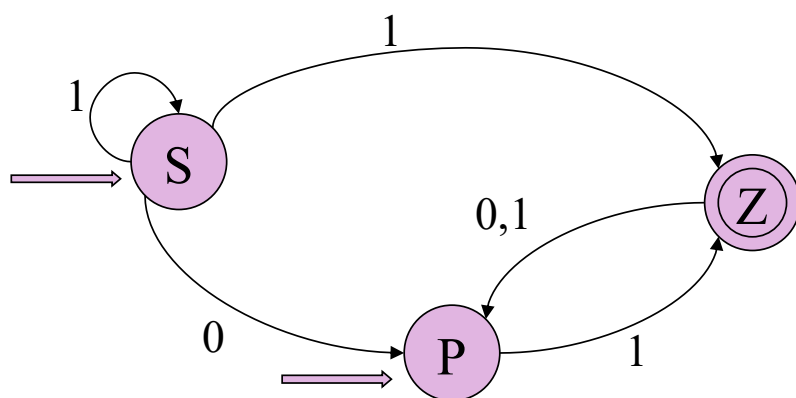


图 (2) 状态转换图

$\delta(s, a)$ 值-----状态转换矩阵值。如图 (1) 所示：行--状态, 列--输入字符

一个NFA也可以表示为一张 (确定的) 状态转换图, 如图 (2) 所示

NFA--练习

画出下列NFA的状态转换矩阵和状态转换图

NFA $M = (\{S, Q, U, V, Z\}, \{0, 1\}, \delta, \{S\}, \{Z\})$;

$$\delta(S, 0) = \{V, Q\} \quad \delta(S, 1) = \{U, Q\}$$

$$\delta(U, 0) = \Phi \quad \delta(U, 1) = \{Z\}$$

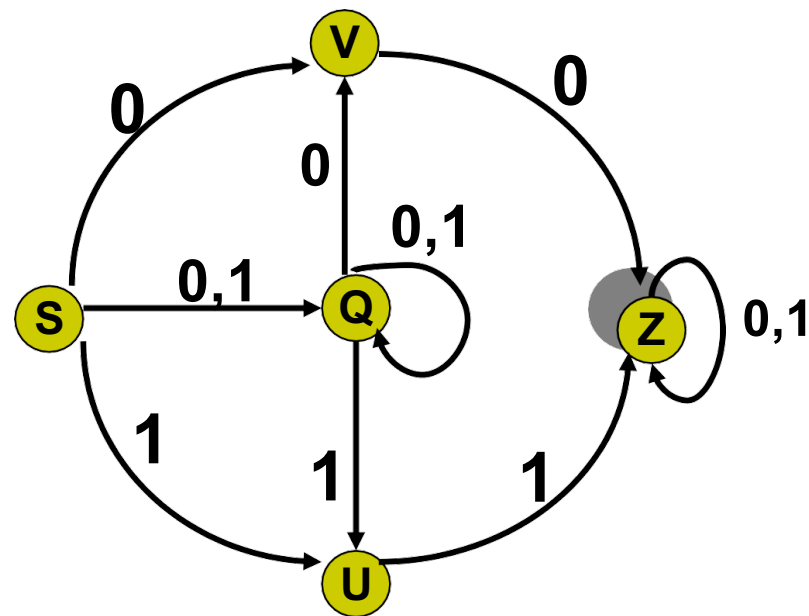
$$\delta(V, 0) = \{Z\} \quad \delta(V, 1) = \Phi$$

$$\delta(Q, 0) = \{V, Q\} \quad \delta(Q, 1) = \{U, Q\}$$

$$\delta(Z, 0) = \{Z\} \quad \delta(Z, 1) = \{Z\}$$

答案

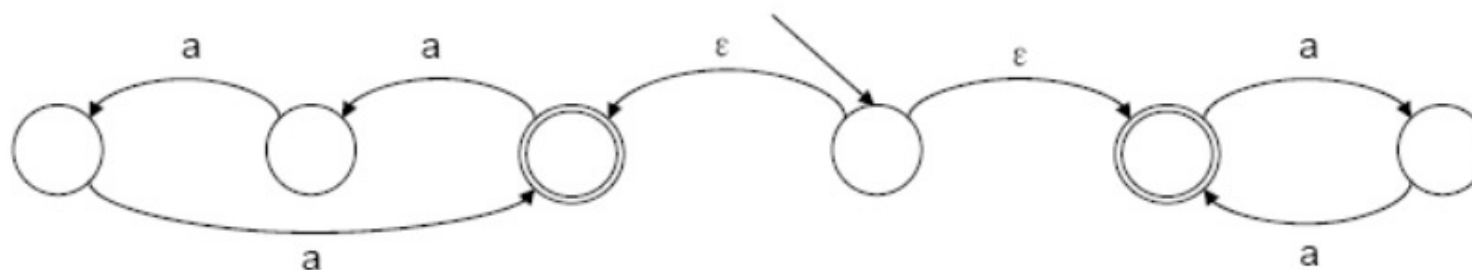
	0	1
S	{V,Q}	{U,Q}
Q	{V,Q}	{U,Q}
U	ϕ	{Z}
V	{Z}	ϕ
Z	{Z}	{Z}



图中某些状态射出两条具有相同标记的弧，如**S,Q**.

非确定有限自动机(NFA)接受的字符串

- 对于 Σ^* 中任何字 α ，若存在一条从某一初态到某一个终态的路径，且这条路径上所有弧的标记字符依序连接成的字符串等于 α ，则称 α 可为 NFA M 所识别（接受）。
- 若 M 的某些结点既是初态又是终态，或存在一条从某一初态到某一终态的 ϵ 路径，那么空字 ϵ 可为 M 所识别。

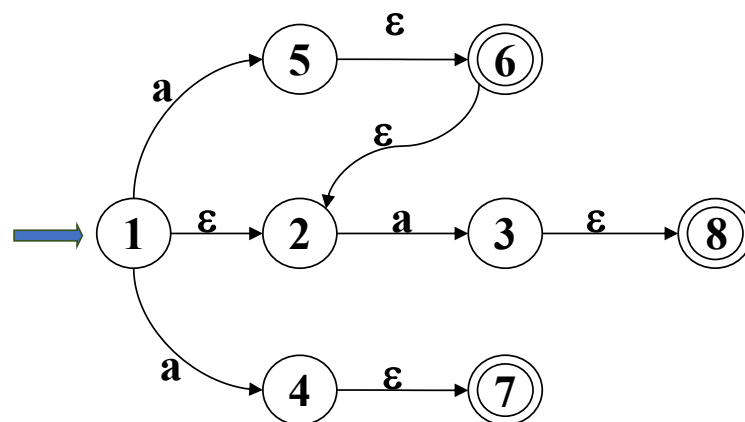


NFA的确定化

NFA的确定化

- 显然，DFA是NFA的特例，但是对于每个NFA M 都存在一个DFA M'' ，使 $L(M)=L(M'')$ 。
- 定义对状态集合 I 的几个有关运算：
 - 状态集合 I 的 ϵ -闭包，表示为 $\epsilon\text{-closure}(I)$ ，定义为一状态集，包括
 - (1)状态集合 I 的任何状态 s 都属于 $\epsilon\text{-closure}(I)$ ；
 - (2)状态集合 I 中的任何状态 s 经任意条 ϵ 弧而能到达的状态的集合。
 - 状态集合 I 的 a 弧转换 $I_a = \epsilon\text{-closure}(\text{move}(I, a)) = \epsilon\text{-closure}(J)$ ，其中 $\text{move}(I, a)$ 为状态集合 J ，其定义是所有那些可从 I 的某一状态经过一条 a 弧而到达的状态的全体。

ϵ -闭包--例子



- $I = \{1\}$, $\epsilon\text{-closure}(I) = \{1, 2\}$;
- $I = \{5\}$, $\epsilon\text{-closure}(I) = \{5, 6, 2\}$;
- $I = \{1, 2\}$,
 $\text{move}(\{1, 2\}, a) = \{5, 3, 4\} = J$;
 $I_a = \epsilon\text{-closure}(\{J\}) = \epsilon\text{-closure}(\{5, 3, 4\}) = \{2, 3, 4, 5, 6, 7, 8\}$;

不具有 ε -转移的NFA转具有 ε -转移的NFA

- 定理1: 对任何一个不具有 ε -转移的NFA $M'=(Q, \Sigma, f', q_0, F')$, 一定存在一个具有 ε -转移的 NFA M , 使

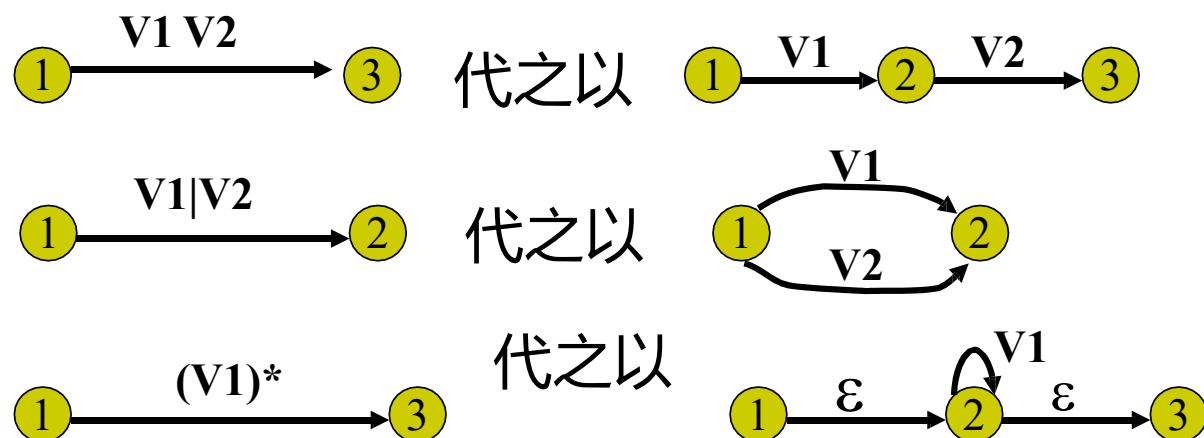
- $L(M) = L(M')$

从 M' 出发构造一个具有 ε -转移的 NFA M , 使得 $L(M) = L(M')$

- 步骤:
 - (1) 引进初态 X 和终态 Y , $X, Y \notin S$, 从 X 到 S_0 中任意状态连接一条 ε 箭弧, 从 F 中任意状态连接一条 ε 箭弧到 Y 。

不具有 ε -转移的NFA转具有 ε -转移的NFA

(2) 对M的状态转换图进一步实施如下替换，其中状态2是新引入状态。重复该过程，直到每条箭弧上标记 ε ，或者为 Σ 中的单个字符。



将 ε -NFA 转化为等价的 DFA

- 定理2: 对于字母表 Σ 上任何一个具有 ε -转移的 NFA M , 一定存在一个的 DFA M' , 使得: $L(M') = L(M)$ 。
- NFA $M = (S, \Sigma, f, S_0, Z')$ 用构造 ε -closure(T) 的方法构造 DFA $M' = (S', \Sigma, f', q_0, Z')$:
- 基本思想:
 - 1) 首先从 S_0 出发, 仅经过任意条 ε 箭弧所能到达的状态所组成的集合 I 作为 M' 的初态 q_0 .
 - 2) 分别把从 q_0 (对应于 M 的状态子集 I) 出发, 经过任意 $a \in \Sigma$ 的 a 弧转换 I_a 所组成的集合作为 M' 的状态, 如此继续, 直到不再有新的状态为止。

将 ε -NFA 转化为等价的 DFA (cont)

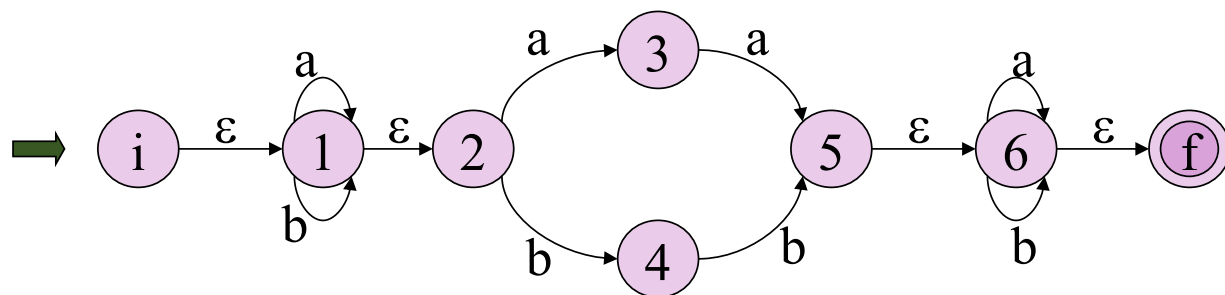
- $I_a = \varepsilon\text{-closure}(\text{move}(I, a)) = \varepsilon\text{-closure}(J)$
- 假定 Σ 有 k 个字母。我们构造一张表，有 $k+1$ 列。
 - 表的每行首列为 $\varepsilon\text{-closure}(X)$, $X \subset \Sigma$ ，且未曾出现在表的第一列。
 - 表的第 $i+1$ 列为 I_{a_i} , a_i 为 Σ 中的第 i 个字符。
 - 重复上述过程填表，因为 Σ 子集有限，所以该过程一定会终止。

■ DFA和NFA的等价性

- 定理3: 对任何一个NFA $M = (S, \Sigma, f, S_0, F)$, 都存在一个DFA $M' = (S', \Sigma, f', s_0, F')$, 使得 $L(M') = L(M)$ 。
 - 证明的思想是由 M 出发构造等价的 M' , 办法是让 M' 的一个状态对应于 M 的一个状态集合。
 - 即若 $\delta(s, a) = \{s_1, s_2, \dots, s_k\}$,
 - 我们将集合 $\{s_1, s_2, \dots, s_k\}$ 作为一个整体看作 M' 中的一个状态, 即 S' 中的一个元素。

NFA确定化的例子

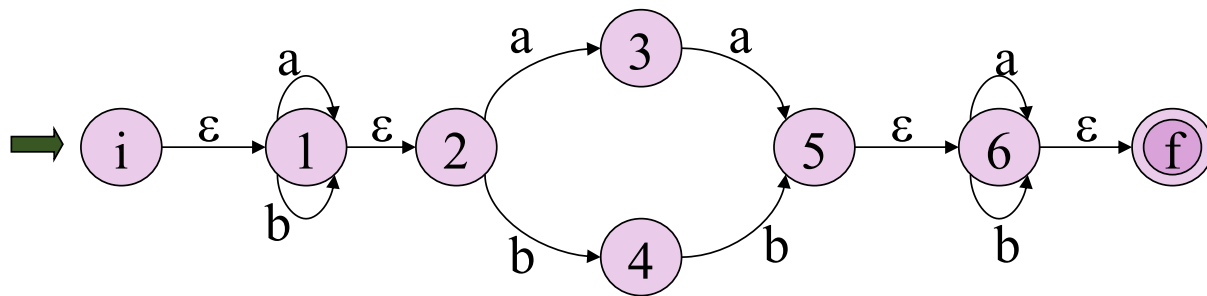
$$I_a = \varepsilon\text{-closure}(\text{move}(I, a)) = \varepsilon\text{-closure}(J)$$



I	Move(I, a)	I_a	Move(I, b)	I_b

NFA确定化的例子

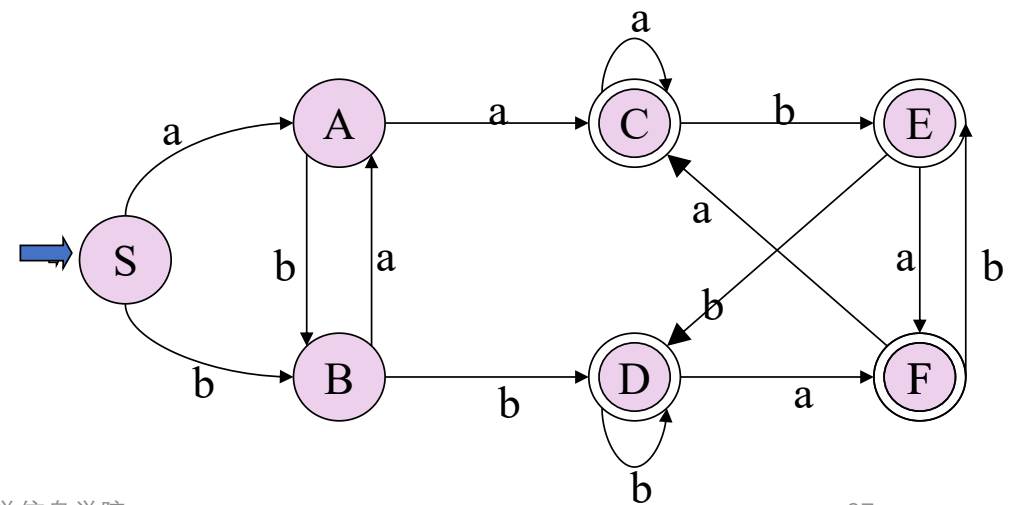
$$I_a = \varepsilon\text{-closure}(\text{move}(I, a)) = \varepsilon\text{-closure}(J)$$



原NFA所有初始状态构成集合的的 ε -闭包作为DFA的初态

包含原终止状态f的状态子集为DFA的终态

	I_a	I_b
$\{i, 1, 2\}$ S	$\{1, 2, 3\}$ A	$\{1, 2, 4\}$ B
$\{1, 2, 3\}$ A	$\{1, 2, 3, 5, 6, f\}$ C	$\{1, 2, 4\}$ B
$\{1, 2, 4\}$ B	$\{1, 2, 3\}$ A	$\{1, 2, 4, 5, 6, f\}$ D
$\{1, 2, 3, 5, 6, f\}$ C	$\{1, 2, 3, 5, 6, f\}$ C	$\{1, 2, 4, 6, f\}$ E
$\{1, 2, 4, 5, 6, f\}$ D	$\{1, 2, 3, 6, f\}$ F	$\{1, 2, 4, 5, 6, f\}$ D
$\{1, 2, 4, 6, f\}$ E	$\{1, 2, 3, 6, f\}$ F	$\{1, 2, 4, 5, 6, f\}$ D
$\{1, 2, 3, 6, f\}$ F	$\{1, 2, 3, 5, 6, f\}$ C	$\{1, 2, 4, 6, f\}$ E



NFA $\rightarrow$ DFA 例子

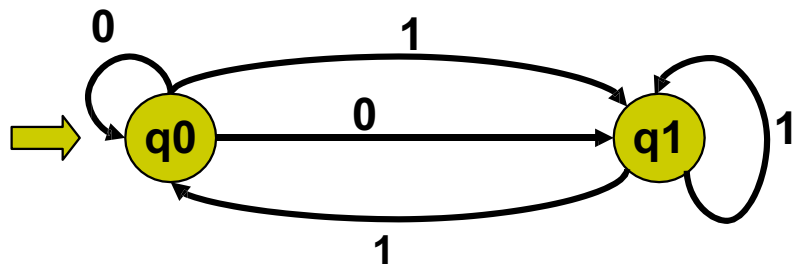
NFA $M = (\{q_0, q_1\}, \{0, 1\}, f, \{q_0\}, \{q_1\})$.

$f(q_0, 0) = \{q_0, q_1\}$ $f(q_0, 1) = \{q_1\}$

$f(q_1, 0) = \Phi$ $f(q_1, 1) = \{q_0, q_1\}$

构造与 M 等价的 DFA M' .

M:



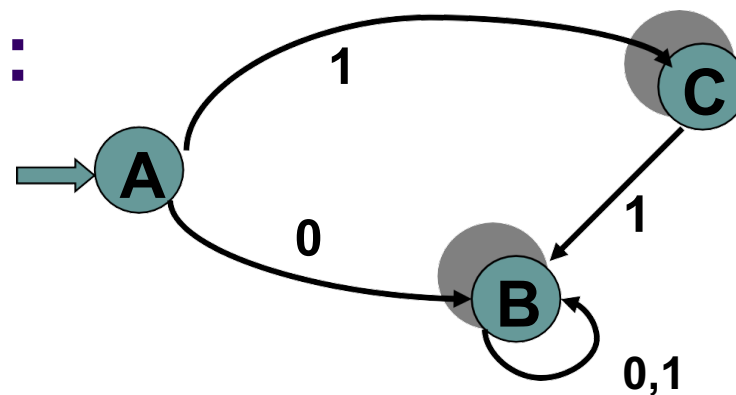
2026/5/27

解:

令 DFA $M' = (\{0, 1\}, Q', f', q_0', F')$

	0	1
A {q0}	B {q0,q1}	{q1} C
B {q0,q1}	B {q0,q1}	{q0,q1} B
C {q1}	Φ	{q0,q1} B

M':



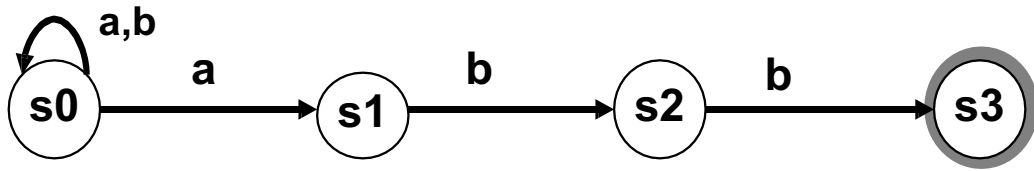
刘爽, 中国人民大学信息学院

38

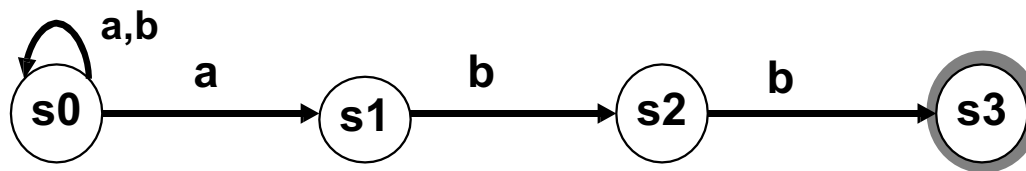
练习

NFA $M = (\{S0, S1, S2, S3\}, \{a, b\}, \{S0\}, \{S3\})$

将M转换成其等价的DFA



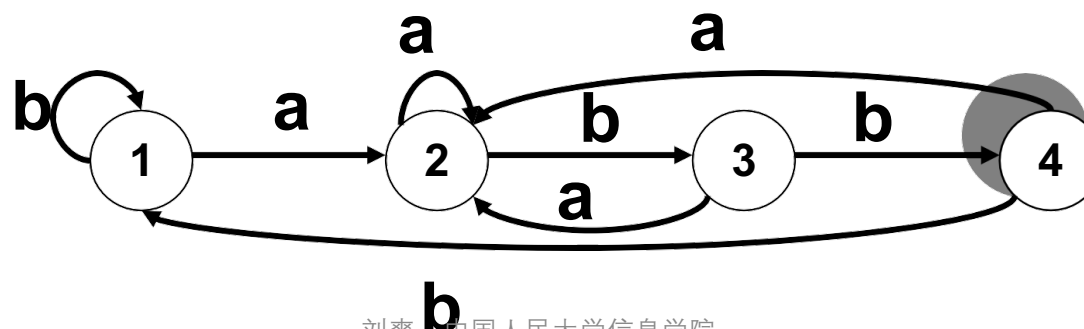
解：



状态转换表：

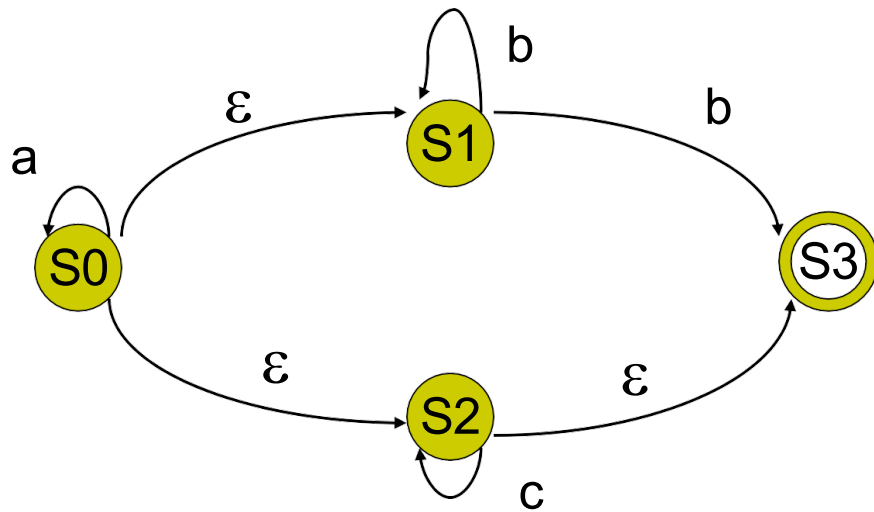
	a	b
1. {s0}	{s0,s1}	{s0}
2. {s0,s1}	{s0,s1}	{s0,s2}
3. {s0,s2}	{s0,s1}	{s0,s3}
4. {s0,s3}	{s0,s1}	{s0}

状态转换图：



■ 练习

- 将下列 ϵ -NFA确定化:



DFA最小化

DFA的最小化

- DFA的最小化(DFA的化简): 一个有穷自动机通过消除多余状态和合并等价状态而转换成一个最小的与之等价的有穷自动机。
 - 寻找一个状态数比M少的 DFA M' , 使得 $L(M) = L(M')$
- 最小状态DFA
 - 不存在多余状态 (死状态)
 - 不存在互相等价 (不可区别) 的两个状态
- 两个状态s和t等价: 从s出发可以读出某个字符串w而到达终态, 从t出发也可以读出字符串w而到达终态; 反之亦然。
- 两个状态s和t可区别: 不满足
 - 兼容性 (一致性) —— 同是终态或同是非终态
 - 传播性 (蔓延性) —— 从状态s出发读入某个a ($a \in \Sigma$) 和从状态t出发读入某个a到达的状态等价。

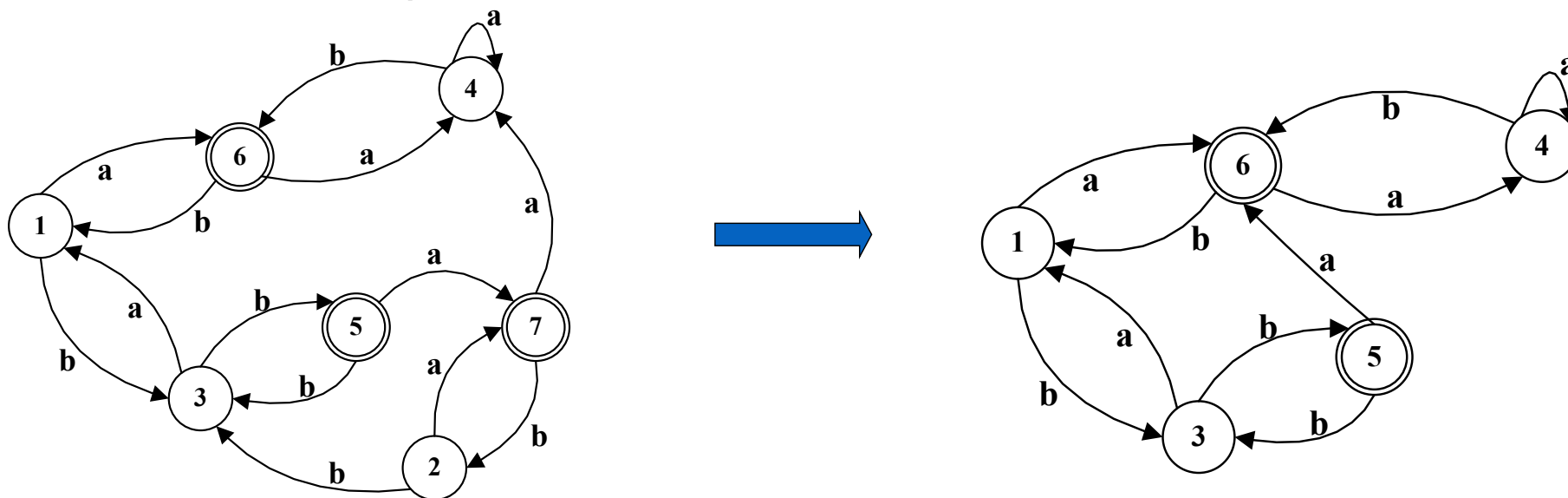
DFA的最小化—算法

- 算法的核心
 - 把M的状态集K分成不相交的子集。
- 结论
 - 接受L的最小状态有穷自动机**不计同构**是唯一的。
- DFA $M = (S, \Sigma, \delta, s_0, S_t)$, 最小状态DFA M'
 - 1.构造状态的初始划分 Π : 终态 S_t 和非终态 $S - S_t$ 两组
 - 2.对 Π 施用传播性原则构造新划分 Π_{new}
 3. 如 $\Pi_{new} == \Pi$,则令 $\Pi_{final} = \Pi$ 并继续步骤4, 否则 $\Pi := \Pi_{new}$ 重复2
 - 4.为 Π_{final} 中的每一组选一代表, 这些代表构成 M' 的状态。若s是一代表,且 $\delta(s,a)=t$, 令r是t组的代表, 则 M' 中有一转换 $\delta'(s, a)=r$ 。 M' 的开始状态是含有 s_0 的那组的代表, M' 的终态是含有 s_t 的那组的代表
 - 5.去掉 M' 中的死状态

DFA的最小化—算法 (cont)

- 对 Π 施用传播性原则构造新划分 Π_{new} 步骤：
 - 假设 Π 被分成 m 个子集 $\Pi = \{S_1, S_2, \dots, S_m\}$, 且属于不同子集的状态是可区别的, 检查 Π 的每一个子集 S_i , 看是否能够进一步划分。
 - 对于某个 S_i , 令 $S_i = \{s_1, s_2, \dots, s_k\}$, 若存在数据字符 a 使得 I_a 不全包含在现行 Π 的某一子集 S_j 中, 则将 S_i 一分为二: 即假定状态 s_1, s_2 , 经过 a 弧分别达到状态 t_1, t_2 , 且 t_1, t_2 属于现行 Π 的两个不同子集, 那么将 S_i 分成两半, 一半含有 s_1 : $S_{i1} = \{s \mid s \in S_i \text{ 且 } s \text{ 经 } a \text{ 弧到达 } t_1 \text{ 所在子集中的某状态}\}$; 另一半含有 s_2 : $S_{i2} = S_i - S_{i1}$
 - 由于 t_1, t_2 是**可区别的**, 即存在 w , 使得 t_1 读出 w 而停于终态, t_2 读不出 w 或读出 w 却未停于终态。因而 aw 可以将 s_1, s_2 区分开。也就是说 S_{i1} 和 S_{i2} 中的状态时可区别的。
 - $\Pi_{\text{new}} = \{S_1, S_2, \dots, S_{i1}, S_{i2}, \dots, S_m\}$

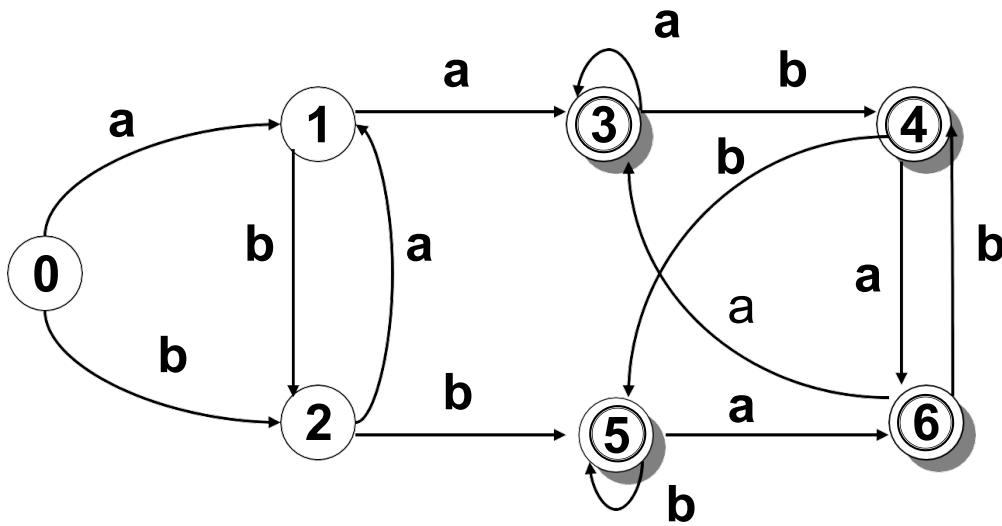
DFA的最小化—例子



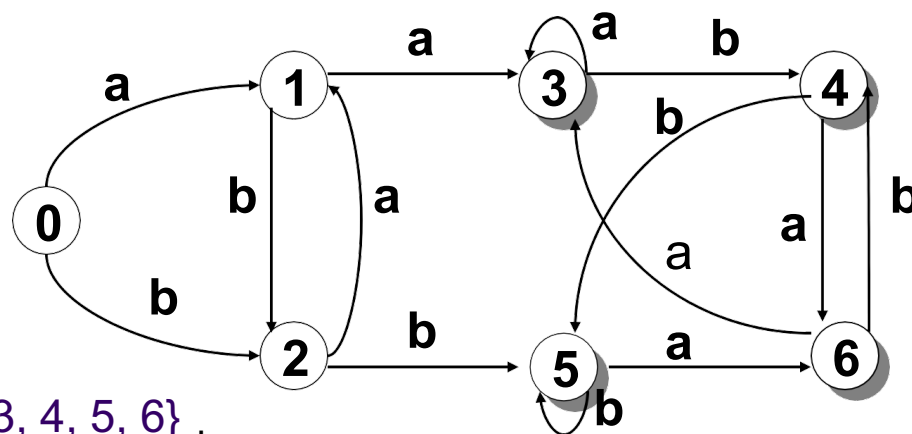
- (1) 初始划分：一个由终态组成，一个由非终态组成 $\Pi_0 = (\{1, 2, 3, 4\}, \{5, 6, 7\})$
- (2) 观察第一个子集，在读入a之后划分为 $\Pi_1 = (\{1, 2\}, \{3, 4\}, \{5, 6, 7\})$
- (3) 观察第二个子集，在读入a之后划分为 $\Pi_2 = (\{1, 2\}, \{3\}, \{4\}, \{5, 6, 7\})$
- (4) 观察第四个子集，在读入a之后划分为 $\Pi_3 = (\{1, 2\}, \{3\}, \{4\}, \{5\}, \{6, 7\})$

练习

- 简化下面的DFA M



解



1) 初始化分 $\Pi_0 = \{\{3, 4, 5, 6\}, \{0, 1, 2\}\}$

2) 观察 Π_0 中的第一个子集: $\{3, 4, 5, 6\}$ $a = \{3, 6\} \subseteq \{3, 4, 5, 6\}$,

$\{3, 4, 5, 6\}$ $b = \{4, 5\} \subseteq \{3, 4, 5, 6\}$ 不能再分。 $\{0, 1, 2\}$ $a = \{1, 3\}$ $\{1, 3\}$ 没有完全包含在 $\{3, 4, 5, 6\}$ 和 $\{0, 1, 2\}$ 中, 故 $\{0, 1, 2\}$ 一分为二。

1 经 a 弧到 3, $3 \in \{3, 4, 5, 6\}$;

0, 2 经 a 弧到 1, $1 \in \{0, 1, 2\}$; 将 $\{0, 1, 2\}$ 分成 $\{0, 2\}$, $\{1\}$

$\Pi_1 = \{\{3, 4, 5, 6\}, \{0, 2\}, \{1\}\}$

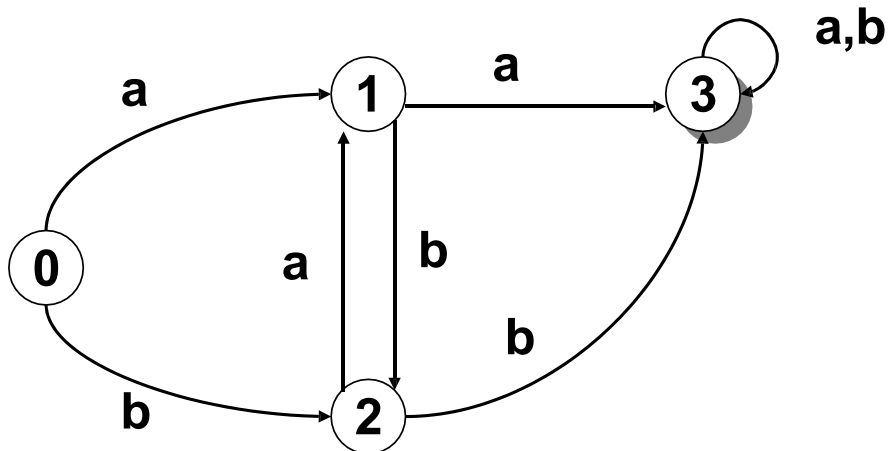
3) 检查 Π_1 中第二个子集 $\{0, 2\}$: $\{0, 2\}$ $b = \{2, 5\}$, $\{2, 5\}$ 不完全落在 $\{0, 2\}$, $\{3, 4, 5\}$ 中, 故 $\{0, 2\}$ 再分成 $\{0\}$, $\{2\}$ 得到

$\Pi_2 = \{\{0\}, \{1\}, \{2\}, \{3, 4, 5, 6\}\}$

4) 检查 Π_2 中的每一个子集, 均不可继续划分, 故 $\Pi_{final} = \Pi_2 = \{\{0\}, \{1\}, \{2\}, \{3, 4, 5, 6\}\}$

解

- 令：3代表{3, 4, 5, 6}，得到：



$$M = (\{0,1,2,3\} , \{a,b\} , f , 0 , \{3\})$$

$$f(0,a) = 1$$

$$f(0,b)=2$$

$$f(1,a) = 3$$

$$f(1,b)=2$$

$$f(2,a) = 1$$

$$f(2,b)=3$$

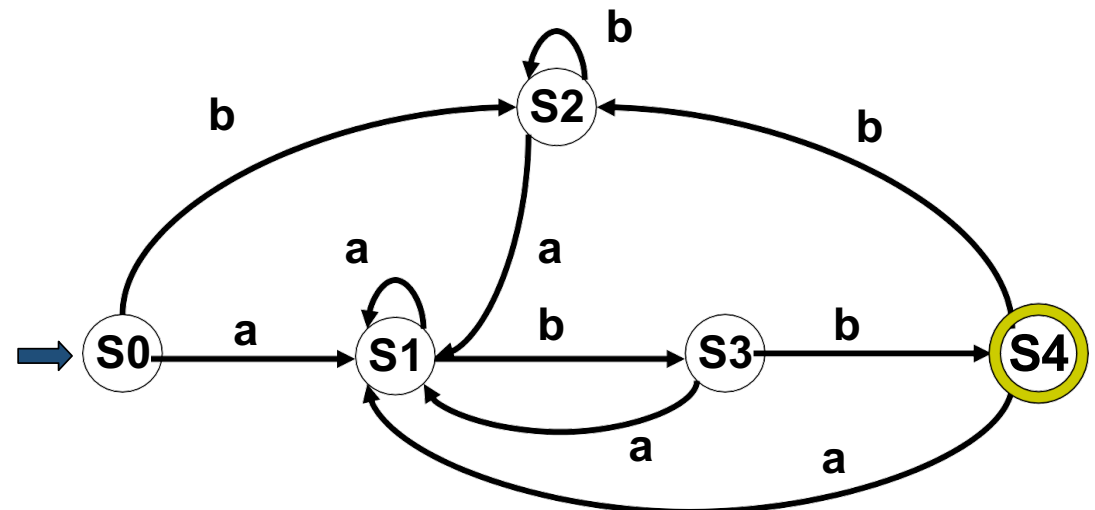
$$f(3,a) = 3$$

$$f(3,b)=3$$

练习

- 将下面的DFA最小化: $M = (\{S0, S1, S2, S3, S4\}, \{a, b\}, f, S0, \{S4\})$

	a	b
S0	S1	S2
S1	S1	S3
S2	S1	S2
S3	S1	S4
S4	S1	S2



解

1) 初始划分: $\{S0, S1, S2, S3\}, \{S4\}$

2) 考察 $\{S0, S1, S2, S3\}$: $\{S0, S1, S2, S3\}a = \{S1\} \subseteq \{S0, S1, S2, S3\}$

$\{S0, S1, S2\}b = \{S2, S3\}$, $\{S3\}b = \{S4\}$ $\{S0, S1, S2, S3\}$ 不包含在同一子集中。一分为二:

NEW: $\{S0, S1, S2\}, \{S3\}, \{S4\}$

3) 考察 $\{S0, S1, S2\}$, $\{S0, S1, S2\}a = \{S1\} \subseteq \{S0, S1, S2\}$

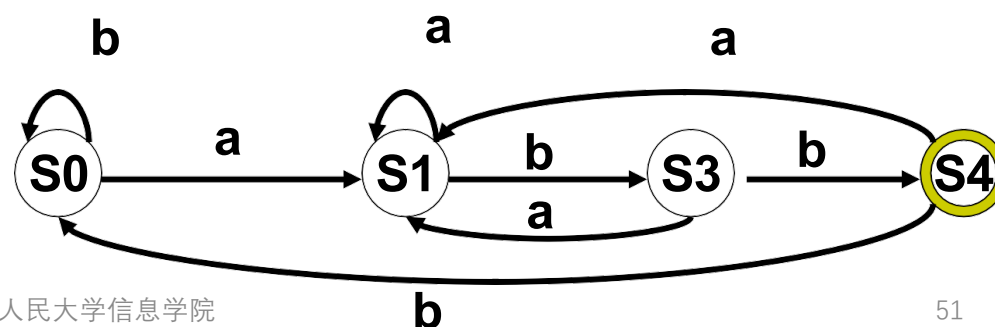
$\{S0, S2\}b = \{S2\}$, $\{S1\}b = \{S3\}$

$\{S0, S1, S2\}$ 不包含在同一子集中。一分为二:

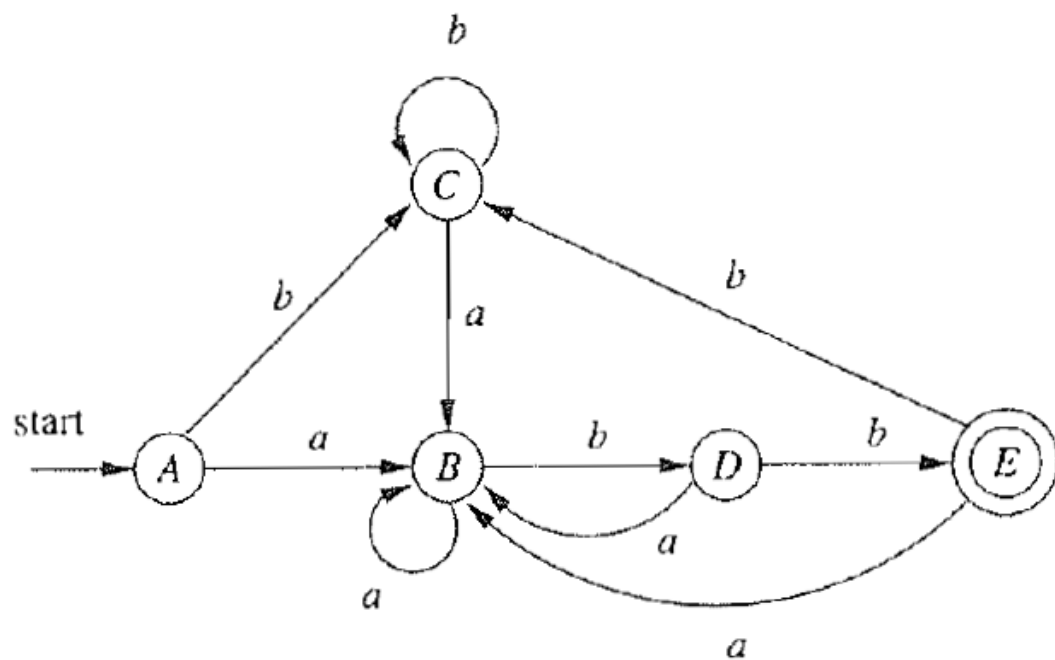
NEW: $\{S0, S2\}, \{S1\}, \{S3\}, \{S4\}$

..... 直到NEW不再改变。

4) $S0$ 作为 $\{S0, S2\}$ 的代表。



■ 练习：最小化下述DFA



■ 内容回顾

- 词法分析器的作用
- 词法分析程序的设计
 - 状态转换图
- 正则表达式和有限自动机
 - 有限自动机 (Finite Automata)
 - 正则表达式 (Regular Expression)
 - 正则文法 (Regular Grammar)
- 词法分析程序的自动生成

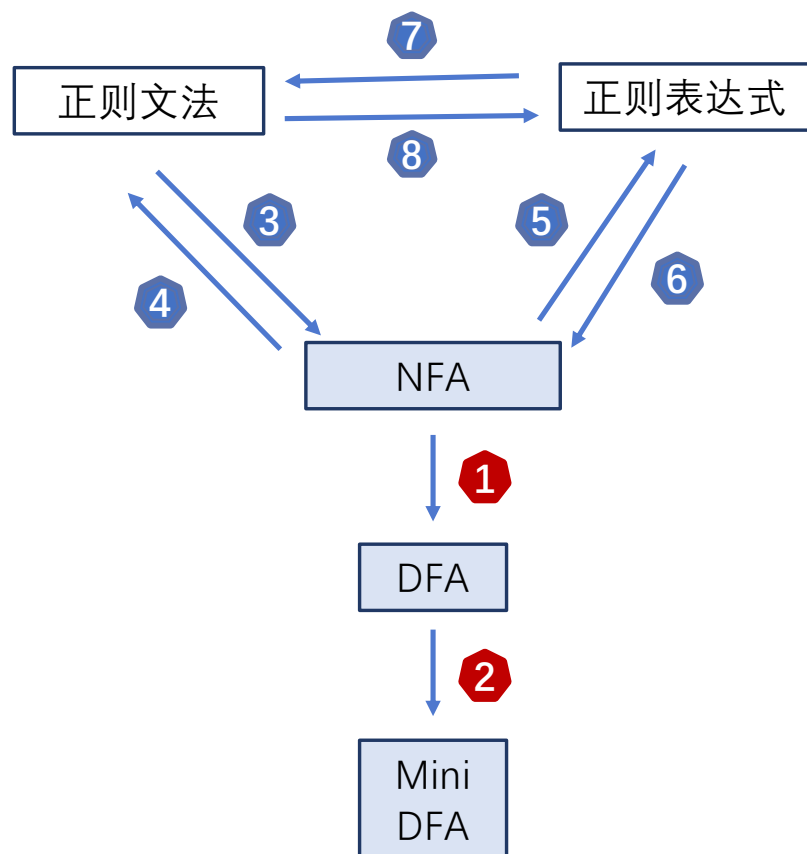


确定有限自动机 (DFA)
非确定有限自动机 (NFA)
NFA确定化
DFA最小化

■ 内容提要

- 词法分析器的作用
- 词法分析程序的设计
 - 状态转换图
- 正则表达式和有限自动机
 - 有限自动机 (Finite Automata)
 - 正则表达式 (Regular Expression)
 - 正则文法 (Regular Grammar)
- 词法分析程序的自动生成

Overview



1. NFA → DFA (NFA 确定化)
2. DFA → mini DFA (DFA 最小化)
3. 正则文法 → NFA (自学)
4. DFA → 正则文法 (自学)
5. NFA → 正则表达式
6. 正则表达式 → NFA
7. 正则表达式 → 正则文法 (自学)
8. 正则文法 → 正则表达式 (自学)



正则表达式和有限自动机

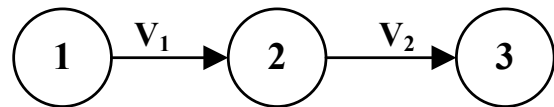
正则表达式和有限自动机的等价性

--有限自动机→正则表达式

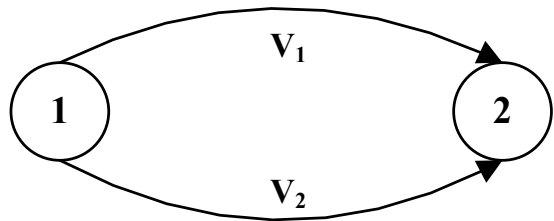
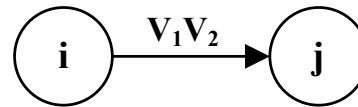
- 关于正则表达式与自动机的等价性，有如下性质：
 - 对任何FA M ，都存在一个正则表达式 r ，使得 $L(r)=L(M)$ 。
 - 对任何正则表达式 r ，都存在一个FA M ，使得 $L(M)=L(r)$ 。

步骤：

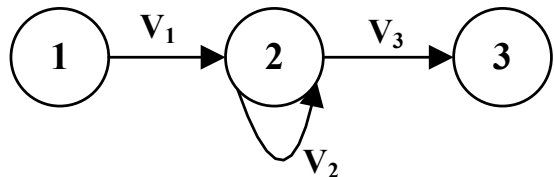
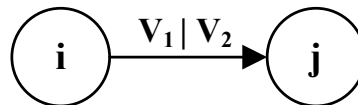
把转换图的意义拓宽,令每条弧上可以标记正则式。



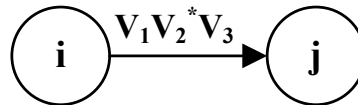
代之以



代之以



代之以

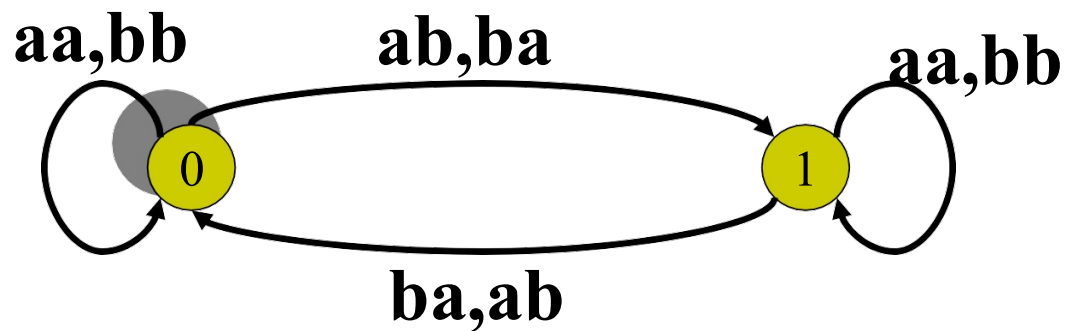


1. 在给定的NFA M 上加两个新状态, 一个为初态 x ,从 x 用 ϵ 弧连接 M 的所有初态,另一为 y ,从 M 的所有终态用 ϵ 弧连到 y ,新的NFA M' 与 M 等价。

2. 对 M' 用左侧等价规则,消除 x,y 以外的其它结。

■ 例子

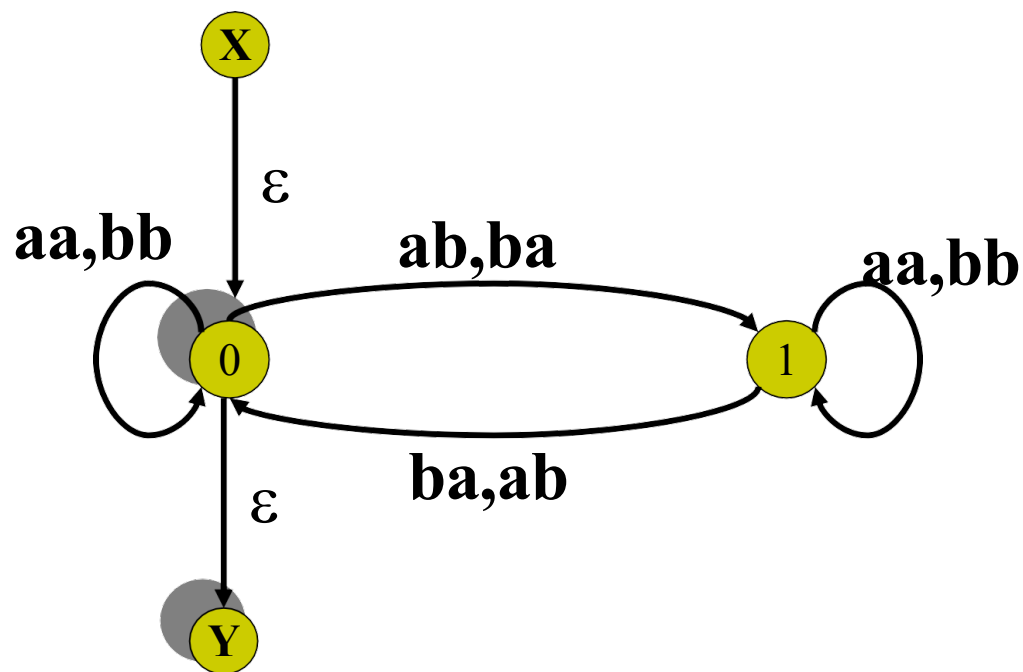
- 把下面的NFA M 转化成等价的正则表达式



NFA M 是识别具有偶数a或偶数个b的非确定有限自动机，则其初始状态是？

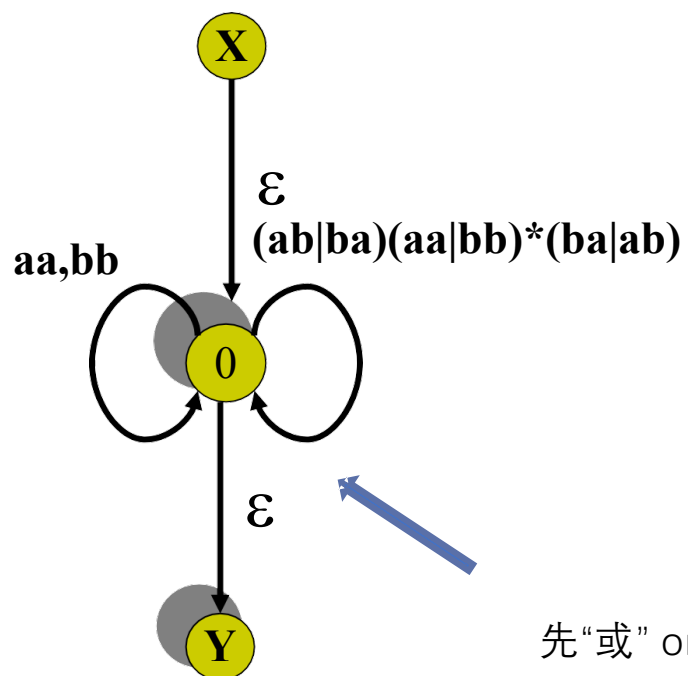
■ 解

1. 在给定的NFA M 上加两个新状态, 一个为初态 X ,从 X 用 ε 弧连接 M 的所有初态,另一为 Y ,从 M 的所有终态用 ε 弧连到 Y , 新的NFA M' 与 M 等价.

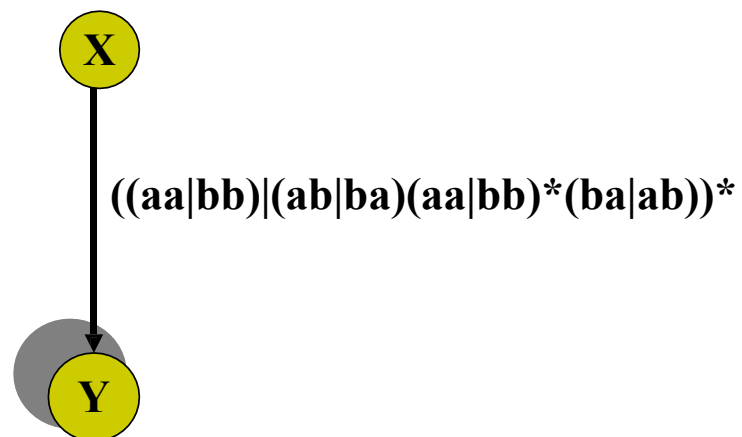
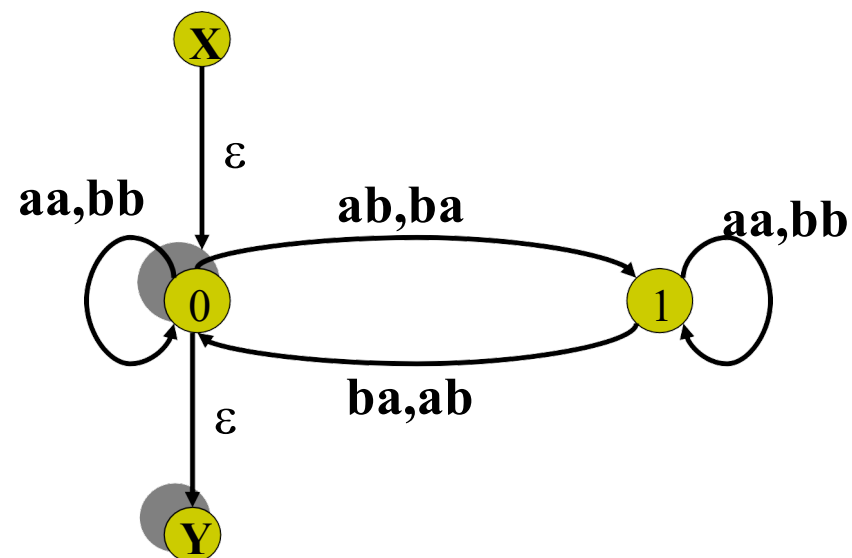


■ 解

2. 对 M' 用等价规则,
消去除 X, Y 以外的其它结点.



先“或” or 先 “*” ?



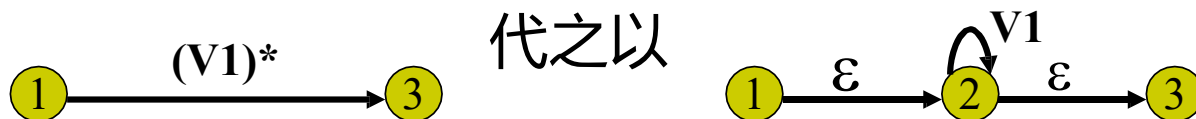
■ 正则表达式和有限自动机的等价性

--正则表达式→有限自动机(I)

1. 对给定的正则表达式构成一个NFA M。先写出：



2. 用以下规则对V进行分解并加进新节点



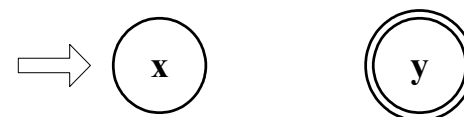
在分解过程中，要求：

- (1) X,Y始终为唯一的初态和终态。
- (2) 所加新结点其名字彼此不同。
- (3) 弧上的标记必须是字符或空字。

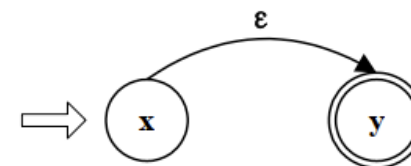
正则表达式和有限自动机的等价性

--正则表达式 $\rightarrow$ 有限自动机(II)

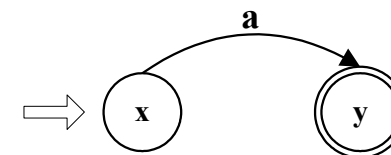
- 对于简单的正则表达式:
 - 对于正则表达式 Φ , 所构造的NFA为:



- 对于正则表达式 ϵ , 构造的NFA为:



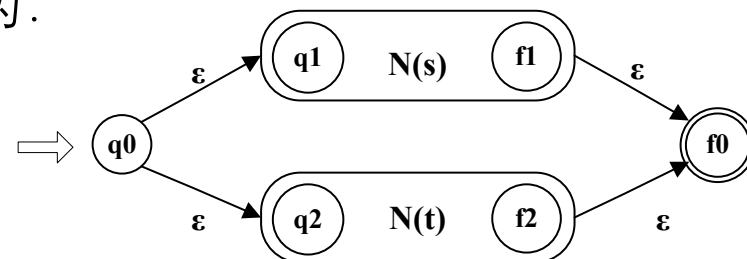
- 对于正则表达式 a , $a \in \Sigma$, 构造的NFA为:



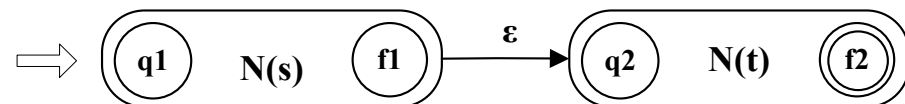
正则表达式和有限自动机的等价性

--正则表达式 $\rightarrow$ 有限自动机

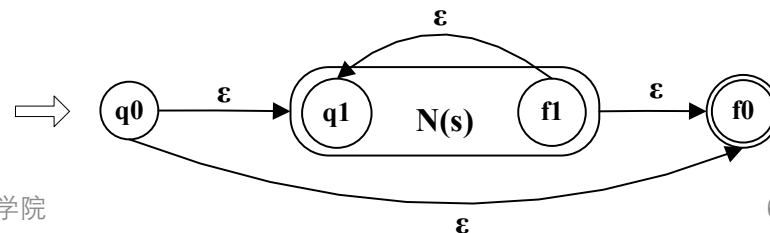
- 若 s, t 为 Σ 上的正则表达式, 相应的NFA分别为 $N(s)$ 和 $N(t)$, 则
 - 对于正则表达式 $R = s \mid t$, 所构造的NFA(R)为:



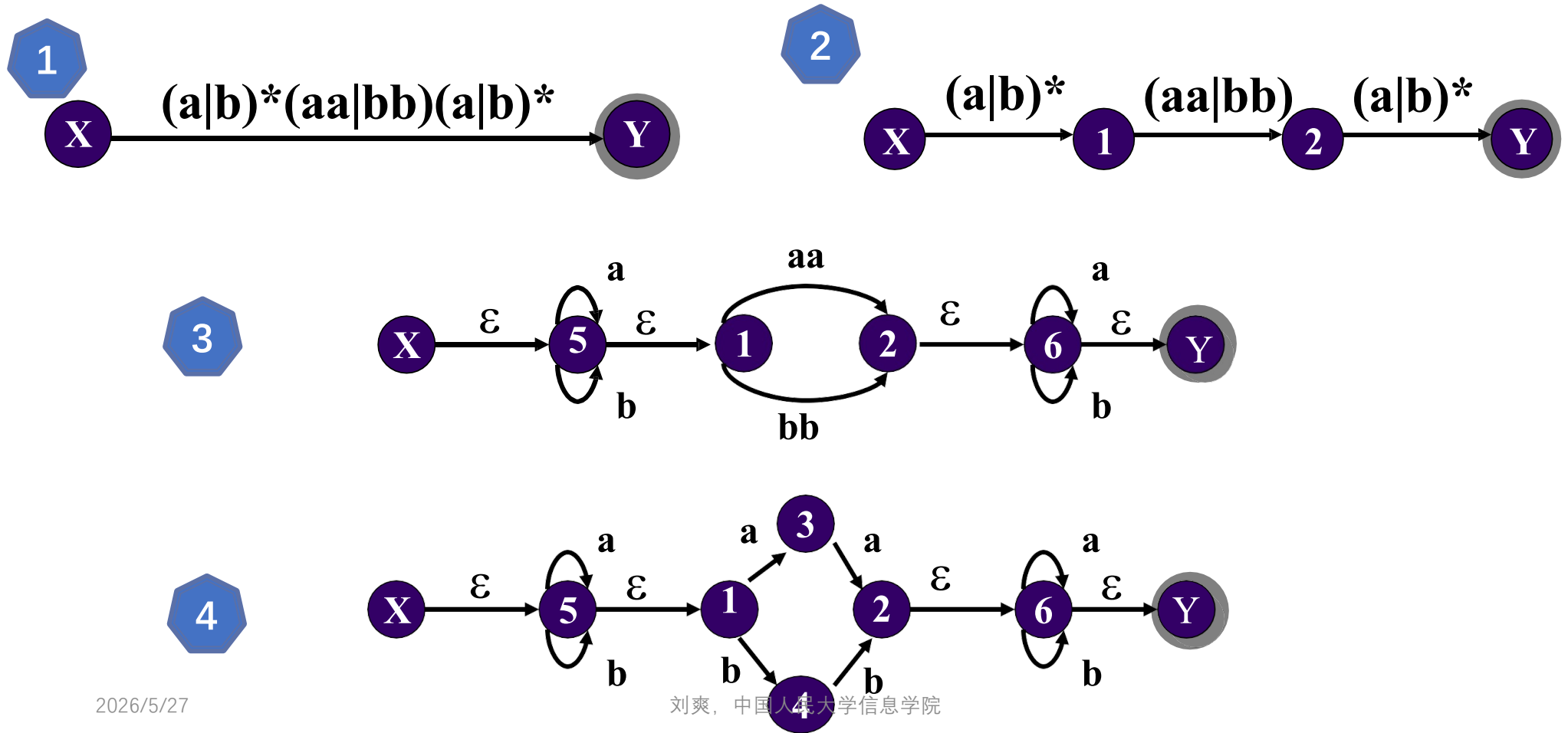
- 对于正则表达式 $R = st$, 构造的NFA(R)为:



- 对于正则表达式 $R = s^*$, 构造的NFA(R)为:



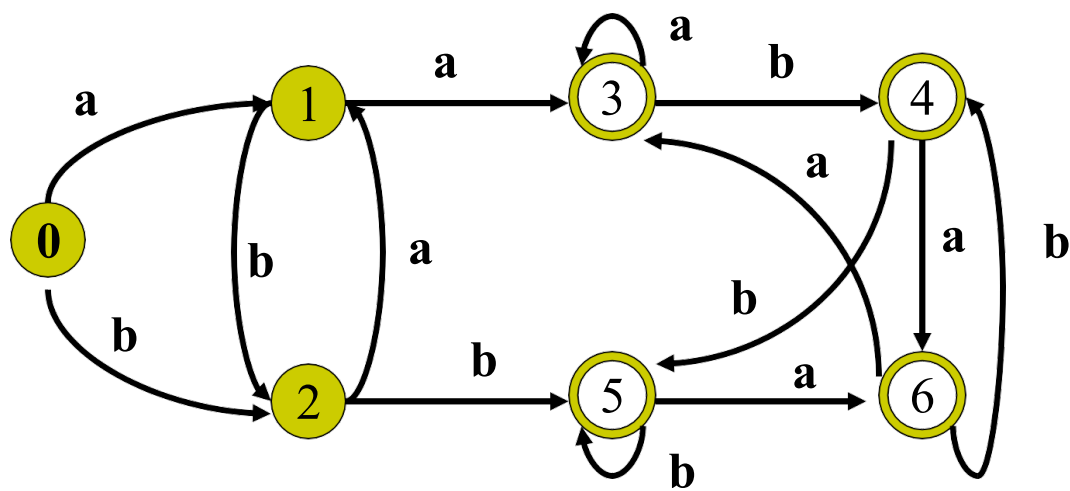
■ 例子 (设 $V = (a|b)^*(aa|bb)(a|b)^*$)



解

I	Ia	Ib
0 {X,5,1}	{5,3,1}	{5,4,1}
1 {5,3,1}	{5,3,1,2,6,Y}	{5,4,1}
2 {5,4,1}	{5,3,1}	{5,3,1,2,6,Y}
3 {5,3,1,2,6,Y}	{5,3,1,2,6,Y}	{5,4,1,6,Y}
4 {5,4,1,6,Y}	{5,3,1,6,Y}	{5,3,1,2,6,Y}
5 {5,4,1,2,6,Y}	{5,3,1,6,Y}	{5,4,1,2,6,Y}
6 {5,3,1,6,Y}	{5,3,1,2,6,Y}	{5,4,1,6,Y}

■ 解

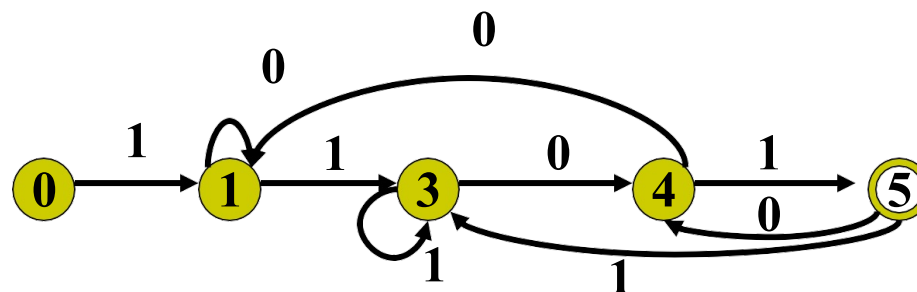


■ 练习

构造下列正则表达式相应的DFA

$1(0|1)^*101$

■ 练习



构造下列正规式相应的DFA

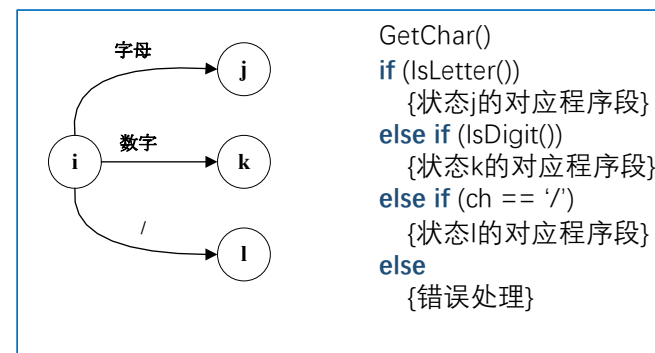
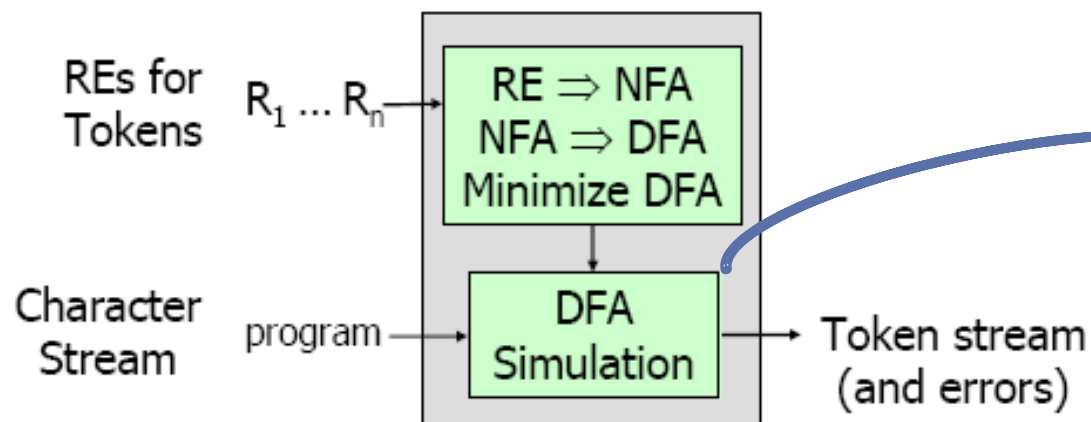
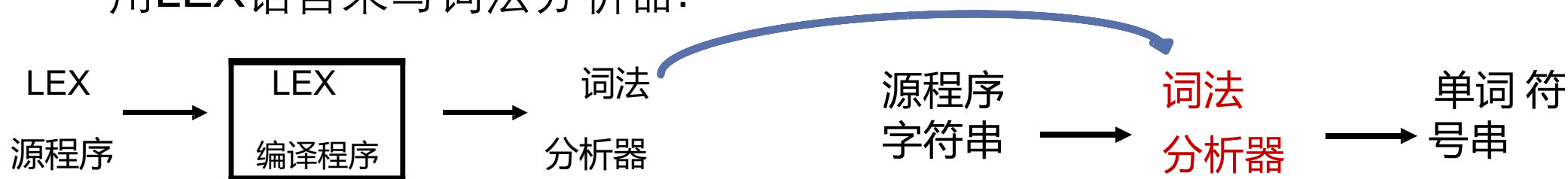
$1(0|1)^*101$

■ 内容提要

- 词法分析器的作用
- 词法分析程序的设计
 - 状态转换图
- 正规表达式和有限自动机
 - 正规表达式 (Regular Expression)
 - 有限自动机 (Finite Automata)
- 词法分析程序的自动生成

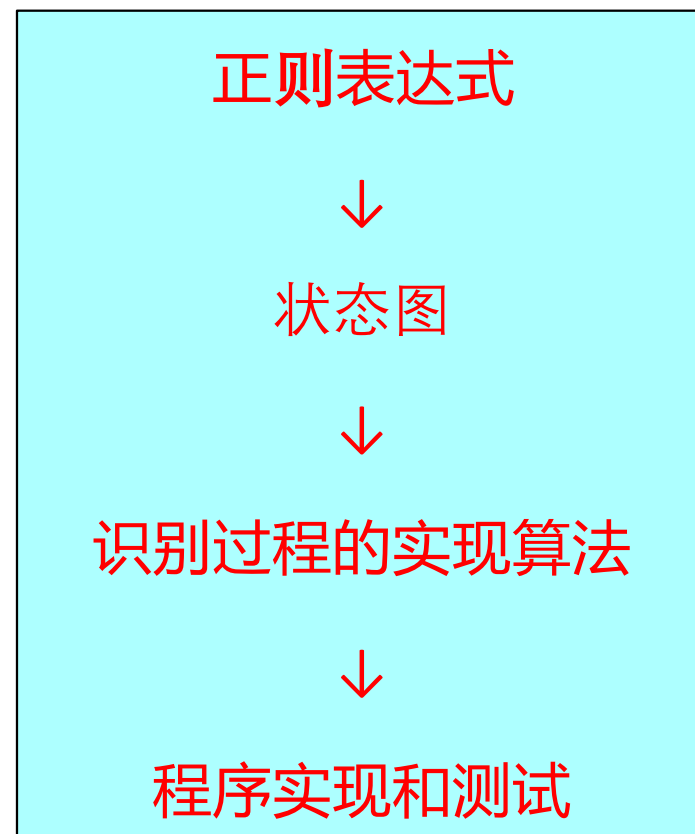
词法分析器的自动生成

- 用LEX语言来写词法分析器:



■ 词法分析器的实现步骤

- 用正则表达式描述词法规则，设置单词种别和属性；
- 按照右图所示步骤，逐步实现词法分析器；

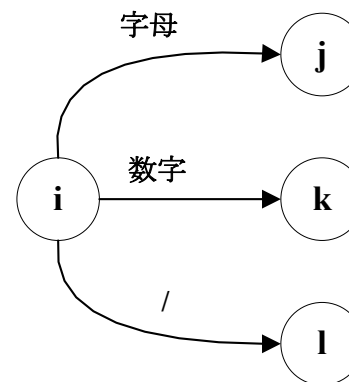


■ 状态转换图的实现

- 程序实现:每个状态结点对应一段程序.

1) 不含回路的分叉结点:

- CASE 或者 IF--THEN—ELSE



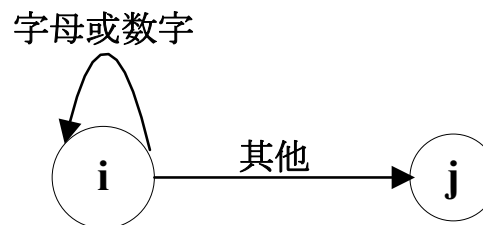
```
GetChar()
if (IsLetter())
    {状态j的对应程序段}
else if (IsDigit())
    {状态k的对应程序段}
else if (ch == '/')
    {状态l的对应程序段}
else
    {错误处理}
```

2) 含回路的分叉结:

- WHILE

3) 终点结:

- RETURN(Code, Value): 返回调用者



```
GetChar();
while (IsLetter() or IsDigit())
    GetChar();
    状态j的对应程序段
```

■ LEX语言的一般介绍:

LEX程序由两部分生成:

(1) 正则式 (辅助) 定义式

由一些LEX语句组成,形式为:

其中: r_i 为正则式,它定义在 $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$. d_i 为其简名

$d_1 \rightarrow r_1$
 $d_2 \rightarrow r_2$
...
 $d_m \rightarrow r_m$

(2) 识别规则

LEX语句组成:

P_i 是词形,由定义在 $\Sigma \cup \{d_1, d_2, \dots, d_n\}$ 上的正则表达式表示.

A_i 是动作,当识别出 P_i 后应做的工作.

$P_1: \quad \{A_1\}$
 $P_2: \quad \{A_2\}$
...
 $P_n: \quad \{A_n\}$

例子

letter	→	A B C ... Z
digit	→	0 1 2 ... 9
iden	→	(letter digit)*

IF:	{return(1, -)}
DO:	{return(2, -)}
Iden:	{return(3, getEntry())}
=:	{retrun(4, -)}
...	

■ LEX工作过程

- 首先，使用LEX语言写一个定义词法分析器的源程序lex.l。
- 然后利用LEX编译器将lex.l转换成C语言程序lex.yy.c。它包括从lex.l的正则表达式构造的状态转换表以及使用该表格识别词素的标准子程序。
- 与lex.l中正则表达式相关联的动作是C代码段，这些动作可以直接加入到lex.yy.c中。
- 最后，lex.yy.c通过C编译器生成目标程序，这个目标程序就是把输入流转换成记号序列的**词法分析器**。

■ Lex词法分析器如何工作:

P1: {A1}
P2: {A2}
...
Pn: {An}

1) 最长匹配原则:

L扫描输入串,寻找最长的子串匹配某一个 P_i ; 并把该子串截下放入TOKEN缓冲区; 然后调用动作 A_i ,把表示 P_i 对应的二元式送给语法分析器.

2) 优先匹配原则:

- 在服从最长匹配的前提下,处于前面的 P_i ,匹配优先权就越高.
- 解决二义性问题

3) 出错处理:

在输入串中找不到与某一个 P_i 匹配的子串,则要报告出错.

4) A_i 返回单词的种别和内部值。在LEX程序中用 RETURN(C, LEXVAL)

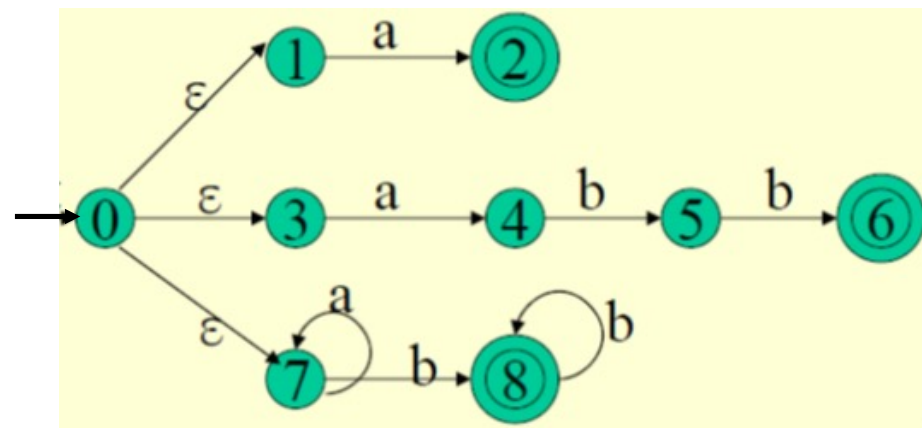
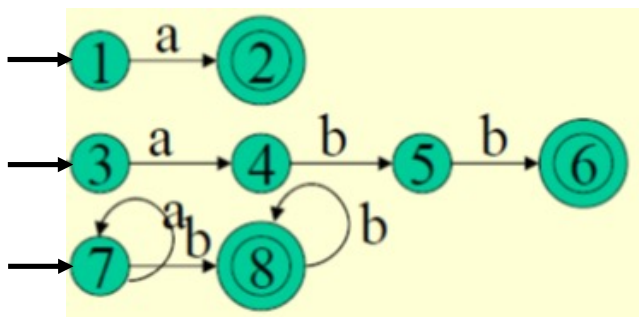
LEX举例

例子:

Lex 源程序

a	{ }
abb	{ }
a*bb*	{ }

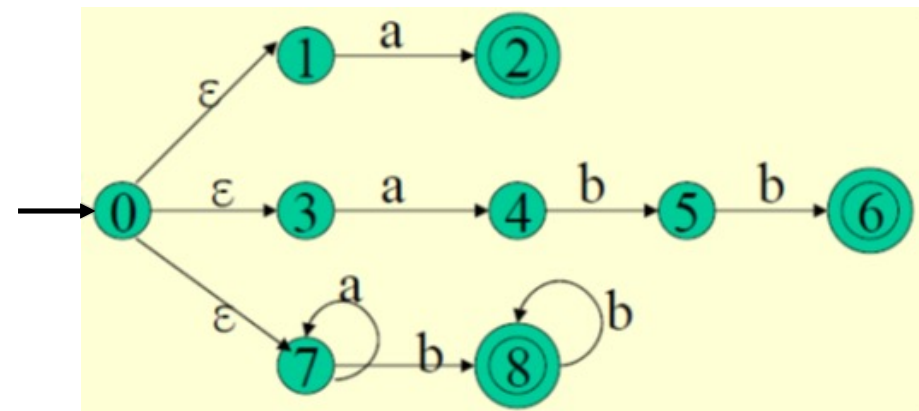
读入Lex源程序, 生成NFA, 合并成一个NFA,
确定化最小化



LEX举例

- NFA确定化: 给出状态转换表

状态	a	b	到达终态所识别的单词
0137	247	8	
247	7	58	a
8	-	8	a*bb*
7	7	8	
58	-	68	a*bb*
68	-	8	abb





正则文法和有限自动机

正则文法

文法 $G = (V_T, V_N, P, S)$ V_T 是非空有限集, 每个元素是一个终结符。
 V_N 是非空有限集, 每个元素是一个非终结符. S 是一个非终结符, 是开始符号 (S 在产生式的左端必须至少出现一次), P 是产生式的集合。

- 如果 P 中的每一个产生式的形式为 (其中, $A, B \in V_N, a \in V_T \cup \{\epsilon\}$)

$A \rightarrow aB$ 或 $A \rightarrow a$, 则称 G 是右线性文法。

- 若文法 G 中的每一个产生式的形式为

$A \rightarrow Ba$ 或 $A \rightarrow a$, 则称 G 是左线性文法。

右线性文法和左线性文法都称为正则文法 (3型文法), 其所产生的语言都称为正则语言 或 3型语言。

■ 正则文法与有限自动机的等价性

- 对于正则文法 G 和有限自动机 M ，如果 $L(G)=L(M)$ ，则称 G 和 M 是等价的。关于正则文法和有限自动机的等价问题，有以下结论：
 - 对每一个右线性正则文法 G_R 或左线性正则文法 G_L ，都存在一个有限自动机(FA) M ，使得 $L(M) = L(G_R) = L(G_L)$ 。
 - 对每一个有限自动机(FA) M ，都存在一个右线性正则文法 G_R 和左线性正则文法 G_L ，使得 $L(M) = L(G_R) = L(G_L)$ 。

正则文法和有限自动机的等价性

--右线性文法 $\rightarrow$ 有限自动机

- 右线性文法 $G = (V_N, V_T, S_G, P)$, 构造 $M = (S, \Sigma, \delta, S_0, F)$:
 - 令 $S = V_N \cup \{f\}$, f 为一新增非终结符号且 f 不属于 V_N ;
 - 令 $\Sigma = V_T$;
 - 令 $S_0 = \{S_G\}$;
 - 令 $F = \{f\}$;
 - 令 δ 如下:
 - 对于 G 中每一形如 $A \rightarrow aB$ 的产生式, 其中 $A, B \in V_N$ 且 $a \in V_T \cup \{\epsilon\}$, 从结点 A 引一条箭弧到结点 B , 并用符号 a 标记这条箭弧, 即 $\delta(A, a) = B$.
 - 对于 G 中每一形如 $A \rightarrow a$ 的产生式, 其中 $A \in V_N$ 且 $a \in V_T \cup \{\epsilon\}$, 从结点 A 引一条箭弧到终态结点 f , 并用符号 a 标记这条箭弧, 即 $\delta(A, a) = f$.

在 G 中, $S_G \xRightarrow{+} w$ 的充要条件是: 在 M 中, 从状态 S_0 出发到 f 有一条通路, 其上所有箭弧的标记符号依次连接起来恰好等于 w , 这就是说 $w \in L(G)$ 当且仅当 $w \in L(M)$, 故 $L(G) = L(M)$

■ 例题

设给定右线性文法 $G[S]$:

$$\begin{aligned} S &\longrightarrow aS | bA | b \\ A &\longrightarrow aS \end{aligned}$$

构造与其等价的有限状态自动机

■ 例题

设给定右线性文法G: $S \longrightarrow aS|bA|b$
 $A \longrightarrow aS$

构造与其等价的有限状态自动机

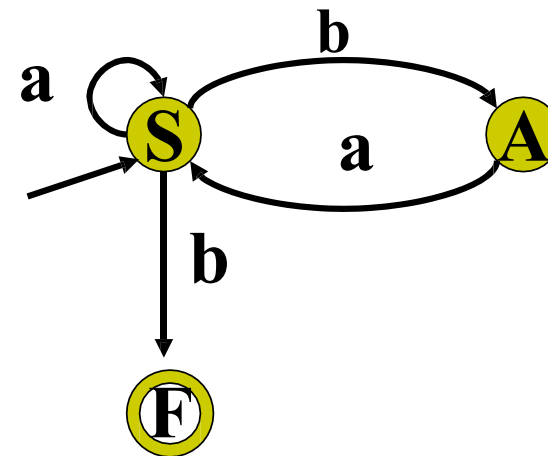
$M = (\{S, A, f\}, \{a, b\}, \delta, \{S\}, \{f\})$

$\delta(S, a) = S,$

$\delta(S, b) = A,$

$\delta(S, b) = f,$

$\delta(A, a) = S$



正则文法和有限自动机的等价性

--左线性文法→有限自动机

- $G = \{V_N, V_T, P, S_G\}$ 是一个左线性文法, 构造的状态转换图 $M = (S, \Sigma, \delta, S_0, F)$ 如下:
 - 令 $S = V_N \cup \{q_0\}$, q_0 为一新增非终结符号且 q_0 不属于 V_N ;
 - 令 $\Sigma = V_T$;
 - 令 $S_0 = \{q_0\}$;
 - 令 $F = \{S_G\}$;
 - 令 δ 如下:
 - 对于 G 中每一形如 $A \rightarrow Ba$ 的产生式, 其中 $A, B \in V_N$ 且 $a \in V_T \cup \{\epsilon\}$, 从结点 B 引一条箭弧到结点 A , 并用符号 a 标记这条箭弧, 即 $\delta(B, a) = A$
 - 对于 G 中每一形如 $A \rightarrow a$ 的产生式, 其中 $A \in V_N$ 且 $a \in V_T \cup \{\epsilon\}$, 从初态 q_0 引一条箭弧到结点 A , 并用符号 a 标记这条箭弧, $\delta(q_0, a) = A$

■ 例题

- 对于左线性文法 $G[S]: (\{S, U\}, \{0, 1\}, P, S)$ 其中 $P = \{S \rightarrow S1, S \rightarrow U1, U \rightarrow U0, U \rightarrow 0\}$

构造与其等价的有限自动机

■ 例题

- 对于左线性文法 $G = (\{S, U\}, \{0, 1\}, P, S)$ 其中 $P = \{S \rightarrow S1, S \rightarrow U1, U \rightarrow U0, U \rightarrow 0\}$

构造与其等价的有限自动机

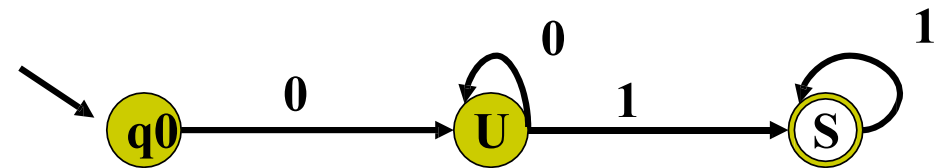
$M = (\{S, U, q_0\}, \{0, 1\}, \delta, \{q_0\}, \{S\})$

$\delta(S, 1) = S,$

$\delta(U, 1) = S,$

$\delta(U, 0) = U,$

$\delta(q_0, 0) = U$



正则文法和有限自动机的等价性

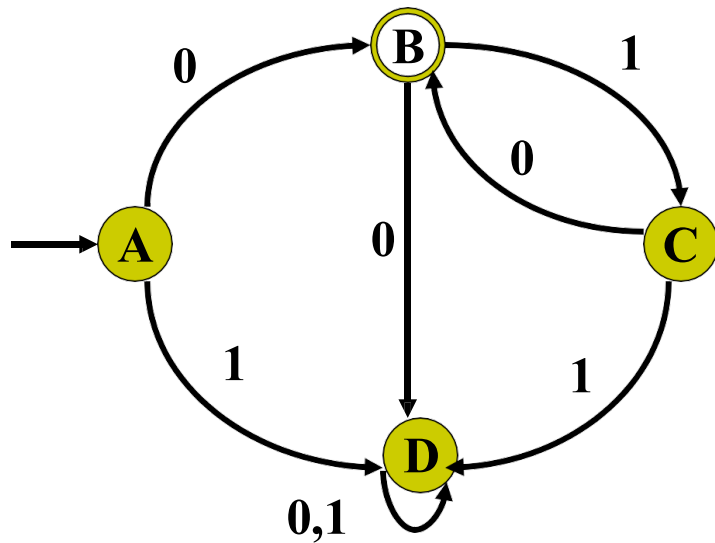
--有限自动机 $\rightarrow$ 右线性文法

- 由DFA $M = (S, \Sigma, \delta, s_0, F)$ 定义右线性文法 $G = (V_N, V_T, S_G, P)$:
 - 如果 s_0 不属于 F , 令 $V_N = S, V_T = \Sigma, S_G = s_0, P$ 是按下面规则定义: 对任何 $a \in \Sigma$ 且 $A, B \in S$, 若有 $\delta(A, a) = B$
 - 当 B 不属于 F , 令 $A \rightarrow aB$;
 - 当 $B \in F$, 令 $A \rightarrow a|aB$
- 如果 $s_0 \in F$, 因为 $\delta(s_0, \varepsilon) = s_0$, 所以, $\varepsilon \in L(M)$, 但 ε 不属于上面构造的 G 所产生的语言 $L(G)$ 。实际上 $L(G) = L(M) - \{\varepsilon\}$, 因而对上面由 M 出发所构造的右线性正规文法 G 中添加一个非终结符号 S'_0 不属于 V_N (即 $V_N = S \cup S'_0$) 和产生式 $S'_0 \rightarrow s_0 | \varepsilon$ (即 $P = P \cup \{S'_0 \rightarrow s_0 | \varepsilon\}$) 并用 $S_G = S'_0$ 代替 s_0 作开始符号。这样经过修正后的 G 仍是右线性正规文法且 $L(G) = L(M)$ 。

类似的从 M 出发可构造左线性文法。

■ 例题:

设DFA $M = (\{A, B, C, D\}, \{0, 1\}, f, A, \{B\})$



$$L(M) = 0(10)^*.$$

构造与其等价的右线性正规文法:

解

$G = (\{A, B, C, D\}, \{0, 1\}, A, P)$
 其中P为产生式的集合:

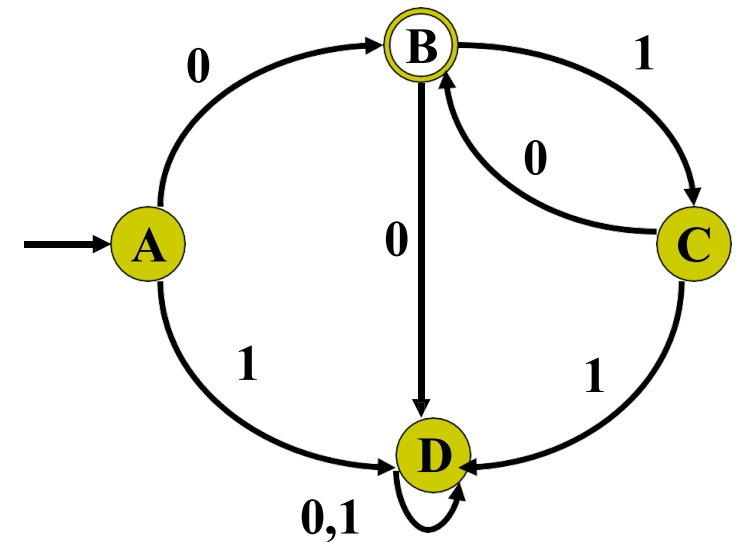
$A \rightarrow 0|0B|1D$

$C \rightarrow 0|0B|1D$

$B \rightarrow 0D|1C$

$D \rightarrow 0D|1D$

$L(G) = L(M) = 0(10)^*$



■ 例题：由右线性正规文法**G**出发构造**NFA**

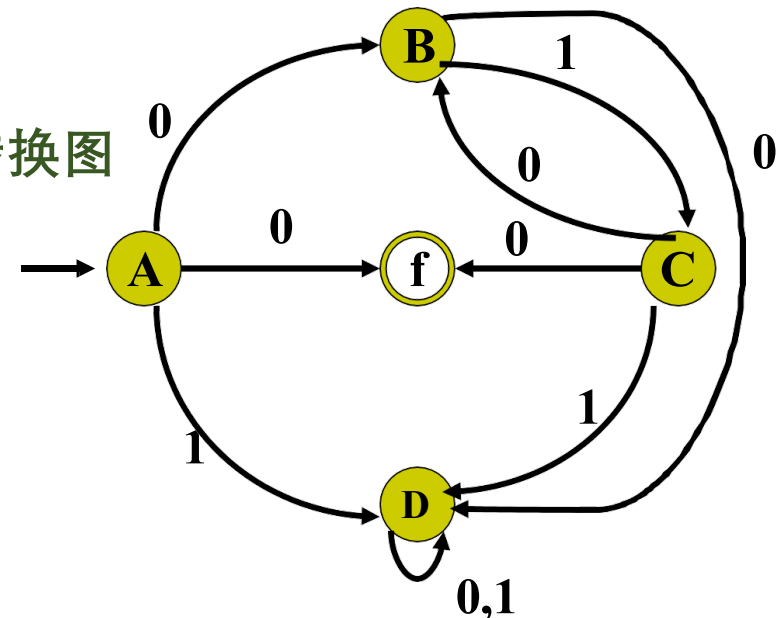
$A \rightarrow 0|0B|1D$ $B \rightarrow 0D|1C$
 $C \rightarrow 0|0B|1D$ $D \rightarrow 0D|1D$

$M' = (\{A, B, C, D, f\}, \{0, 1\}, \delta, \{A\}, \{f\})$
 其中

$\delta(A, 0) = \{f, B\}$ $\delta(A, 1) = \{D\}$
 $\delta(B, 0) = \{D\}$ $\delta(B, 1) = \{C\}$
 $\delta(C, 0) = \{f, B\}$ $\delta(C, 1) = \{D\}$
 $\delta(D, 0) = \{D\}$ $\delta(D, 1) = \{D\}$

$$L(M') = L(M)$$

状态转换图



■ 例题：由NFA构造左线性正规文法

- 从 M' 构造左线性文法 G' 的产生式 P 的方法：
 - 若对任何 $S, S' \in \{B, C, D, f\}$ 且 $a \in \{0, 1\}$ 有 $\delta(S, a) = S'$, 则令 $S' \rightarrow Sa$,
 - 若有 $\delta(A, a) = S$ (A 是 M' 的初态), 则令 $S \rightarrow a$,
- $G' = (\{B, C, D, S\}, \{0, 1\}, S, P)$
- 其中 $S=f$ 为下面产生式的集合：

$$S \rightarrow 0|C0 \quad C \rightarrow B1 \quad B \rightarrow 0|C0 \quad D \rightarrow 1|C1|D0|D1|B0$$
- 有 $L(G') = L(M') = L(G) = L(M) = 0(10)^*$

练习

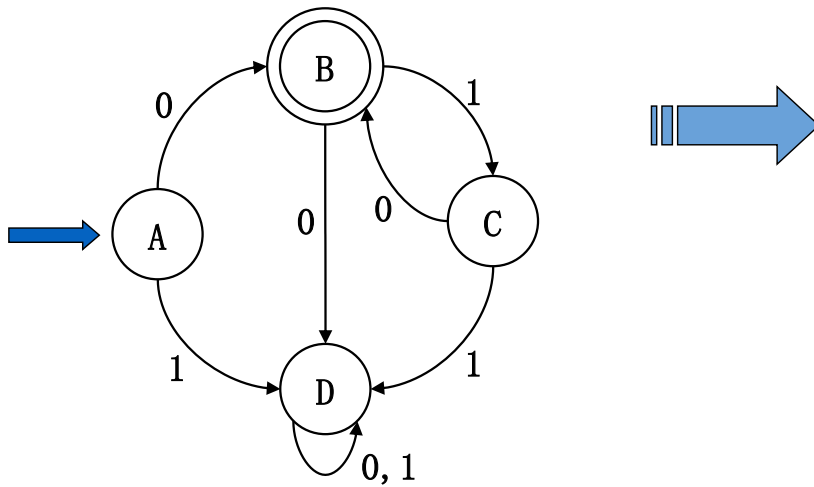


图1 DFA

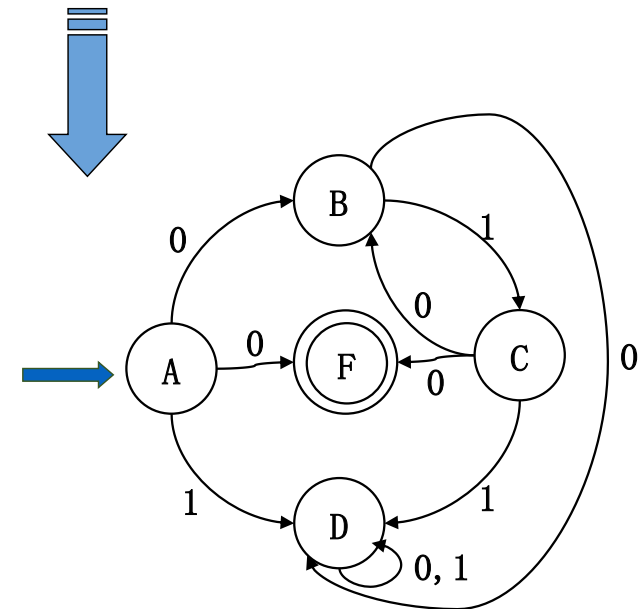
- (1) 由图1的DFA构造与其等价的右线性文法，
- (2) 再从该文法出发构造等价的NFA

正则文法

$G_R = \langle \{0, 1\}, \{A, B, C, D\}, A, P \rangle$ 其中P由以下产生式构成

$A \rightarrow 0 \mid 0B \mid 1D$ $B \rightarrow 0D \mid 1C$

$C \rightarrow 0 \mid 0B \mid 1D$ $D \rightarrow 0D \mid 1D$





正则表达式和正则文法

■ 正则表达式转正则文法 (步骤)

- 对 Σ 上的正则表达式 U , 存在一个正则文法 $G=(V_N, V_T, P, S): L(G)=L(U)$
步骤:

初始: $V_T = \Sigma, S \in V_N$, 生成正则产生式 $S \rightarrow U$

(1) 对形如 $A \rightarrow r_1 r_2$ 的正则产生式: $A \rightarrow r_1 B$

$B \rightarrow r_2 \quad B \in V_N$

(2) 对形如 $A \rightarrow r^* r_1$ 的正则产生式: $A \rightarrow r B$

$A \rightarrow r_1$

$B \rightarrow r B$

$B \rightarrow r_1 \quad B \in V_N$

(3) 对形如 $A \rightarrow r_1 | r_2$ 的正则产生式: $A \rightarrow r_1$

$A \rightarrow r_2$

不断应用上述规则做变换, 直到每个产生式右端只含一个 V_N

正则表达式转正则文法举例

- 例1: $r = a(a \mid d)^*$
 - $V_T = \{a, d\}$ $S \rightarrow a(a \mid d)^*$

- (1) $S \rightarrow aA$
 $A \rightarrow (a \mid d)^*$
- (2) $A \rightarrow (a \mid d)A$
 $A \rightarrow \varepsilon$

- (3) $G[s]: \left. \begin{array}{l} S \rightarrow aA \\ A \rightarrow \varepsilon \\ A \rightarrow aA \\ A \rightarrow dA \end{array} \right\} P$

$V_T = \{a, d\}$
 $V_N = \{S, A\}$
 初始符号为 S

初始: $V_T = \Sigma, S \in V_N$, 生成正则产生式 $S \rightarrow U$

(1) 对形如 $A \rightarrow r_1 r_2$ 的正则产生式:

$A \rightarrow r_1 B$

$B \rightarrow r_2 \quad B \in V_N$

(2) 对形如 $A \rightarrow r^* r_1$ 的正则产生式:

$A \rightarrow rA \quad A \rightarrow r_1$

(3) 对形如 $A \rightarrow r_1 \mid r_2$ 的正则产生式:

$A \rightarrow r_1 \quad A \rightarrow r_2$

不断应用上述规则做变换, 直到每个产生式右端只含一个 V_N

■ 正则文法转正则表达式*

- 对 $G=(V_N, V_T, P, S)$, 存在一个 $\Sigma = V_T$ 上的正则表达式 $r : L(r)=L(G)$

正则文法	正则表达式
规则1 $A \rightarrow xB, B \rightarrow y$	$A=xy$
规则2 $A \rightarrow xA \mid y$	$A=x^*y$
规则3 $A \rightarrow x, A \rightarrow y$	$A=x \mid y$

■ 正则文法转正则表达式举例

- $G[s]:$

$$\begin{aligned} S &\rightarrow aA \\ A &\rightarrow aA \\ A &\rightarrow dA \\ S &\rightarrow a \\ A &\rightarrow a \\ A &\rightarrow d \end{aligned}$$

$$\begin{aligned} \text{规则1 } A &\rightarrow xB, B \rightarrow y & A &= xy \\ \text{规则2 } A &\rightarrow xA \mid y & A &= x^*y \\ \text{规则3 } A &\rightarrow x, A \rightarrow y & A &= x \mid y \end{aligned}$$

$$\begin{aligned} \textcircled{1} \quad S &\rightarrow aA \mid a \\ \textcircled{2} \quad A &\rightarrow aA \mid a \mid dA \mid d \\ \textcircled{3} \quad A &\rightarrow (a \mid d)A \mid (a \mid d) \\ \textcircled{4} \quad A &\rightarrow (a \mid d)^*(a \mid d) \\ S &= a(a \mid d)^*(a \mid d) \mid a = a((a \mid d)^*(a \mid d) \mid \epsilon) = a((a \mid d)^+ \mid \epsilon) \end{aligned}$$

■ 小结

- 词法分析器的作用
- 词法分析程序的设计
 - 状态转换图
- 正规表达式和有限自动机
 - 有限自动机 (Finite Automata)
 - 正则表达式 (Regular Expression)
 - 正则文法 (Regular Grammar)
- 词法分析程序的自动生成

阅读材料：《程序设计语言编译原理（第3版）》，
陈火旺等编著，国防工业出版社，2004年----第三章